

A Systematic Mapping Study of AI Coding Assistants and Agents in Software Engineering: Supply-Side Capabilities versus Demand-Side Empirical Evidence

Empirical Research Project (ERP) 2 by

THIA Boon Sing

School of Computing and Information Systems, Singapore Management University

Supervised by: Prof. TA Nguyen Binh Duong and Prof. ZHU Feida

ABSTRACT

AI systems for software engineering (AI-SE) have evolved rapidly since 2021, from inline autocompletion to multi-step agentic execution. Recent surveys [20, 29, 30] map the supply side of this transition, covering agent capabilities, architectures, and evaluation methods. A parallel conceptual contribution by Treude and Gerosa [26] catalogues eleven developer–AI interaction types along a UI-trigger-mechanism axis, with empirical validation explicitly positioned as future work. What remains missing is a corpus-level synthesis of how software engineers, students, and citizen developers use AI-SE systems in practice. This systematic mapping study addresses that demand-side gap through three research questions. Research Question (RQ)1 asks what the landscape of empirical AI-SE research looks like. RQ2 asks what human-AI interaction modes the literature reports. RQ3 asks where gaps lie between the capabilities emphasized in supply-side surveys and the SDLC activities for which empirical evidence has accumulated. Under a pre-registered protocol, the study screens Scopus and the ACM Digital Library and completes backward snowballing from five seed surveys [2, 19, 20, 29, 30], producing a corpus of primary empirical papers published between 2022 and 2026. Each paper is classified along five facets: Contribution Type, Research Methodology, Population and Context, SDLC Activity, and Agent/Tool Profile. From this corpus the study delivers three linked contributions: a five-facet landscape map for RQ1; a five-mode Human-AI Interaction Mode Taxonomy for RQ2, i.e. Inline Completion, Conversational Prompting, Visual/Declarative Composition, Review and Validation, and Delegated Task Execution, ordered along a delegation-depth axis and supported by mode-layer coding saturation as a dependability check; and a capability–evidence cross-tabulation for RQ3 that converts every under-evidenced or empty intersection into a pre-registered study call within a research agenda for downstream qualitative investigation. The headline finding is that supply-side literature describing what AI systems can do has outpaced demand-side literature documenting how practitioners use them - a pattern we term supply-saturation versus demand-under-evidence. The mismatch is sharpest at early-lifecycle activities (requirements, design), at operational activities (CI/CD, project management), and across the Low-code and No-code paradigms; the research agenda names these intersections as priorities for tool builders, educators, and researchers. To support transparency and computational reproducibility, the complete research repository for this study is available at <https://github.com/alphathia/erp2-submit.git>.

CCS CONCEPTS • Software engineering • Software development techniques • Human-centered computing • Artificial intelligence Agents

Additional Keywords and Phrases: software engineering agents, low-code development agents, no-code development agents, systematic mapping study, AI-assisted coding, human-AI interaction, usage patterns, agentic software engineering

1 INTRODUCTION

1.1 Motivations

Since 2021, AI in software engineering has rapidly progressed from simple, one-line autocompletion to multi-step autonomous agentic execution. While GitHub Copilot popularized inline completion [10] as a seamless workflow integration, the landscape has expanded to include IDE-integrated chat assistants and agentic frameworks. Current commercial agents, such as Cursor, Windsurf, and Claude Code, now move beyond suggestions to propose, plan, execute, and iterate on code across the entire software development lifecycle (SDLC).

Three recent surveys examine the supply side of this technological transition. Wang et al. [29] classify agentic programming systems based on their behavioral dimensions. A second survey by Wang et al. [30] characterizes internal agent architectures through a perception–memory–action framework derived from a review of primary studies. Similarly, Otoum & Elkhaili [20] provide a methods-oriented systematic review of current agentic software engineering techniques. While these surveys collectively describe the capabilities and construction of AI systems, they do not synthesize corpus-level evidence of how professional software engineers, students, or citizen developers utilize these systems in practice.

As a result, a demand-side gap remains in the current literature. Previous technological shifts in software engineering, including Agile, DevOps, and the low-code/no-code platforms recently surveyed by Ajimati et al. [1], consistently demonstrate that technological capability and actual adoption often diverge. A supply-side only perspective cannot address the critical questions of practitioners, educators, and tool-builders regarding the distribution of demand: specifically, where empirical evidence of usage has accumulated, which interaction patterns are being reported, and which intersections of capability and activity remain under-evidenced.

1.2 Research Questions

To structure a demand-side synthesis of the 2022–2026 empirical AI-SE literature, this study poses three research questions:

- **RQ1. landscape:** What is the landscape of empirical research reporting how software engineers, students, and citizen developers use AI coding assistants, SE agents, and AI-augmented low-code and no-code platforms?
- **RQ2. Interaction modes.** What human-AI interaction modes are reported in the empirical literature, and how do they vary across SDLC activities, agent / tool profiles, and population and context?
- **RQ3. Capability-evidence gaps:** Where do gaps exist between the capabilities emphasized in recent supply-side surveys and the SDLC activities for which empirical evidence has accumulated?

1.3 Objectives

The study pursues four research objectives declared in its pre-registered protocol: (i) an evidence landscape map of empirical AI-SE literature along a five-facet classification; (ii) a human–AI interaction mode taxonomy along a delegation-depth axis; (iii) a capability–evidence gap matrix that cross-tabulates supply-side capabilities against empirical SDLC evidence; and finally, an evidence-based research agenda for downstream qualitative investigation. Mapping those objectives to the questions above, the paper contributes:

1. **A 2022–2026 demand-side landscape (RQ1)** that provides a systematic mapping of 640 primary empirical papers published between 2022 and 2026, each classified along: Facet 1 (F1) - Contribution type, Facet 2

(F2) - research methodology, Facet 3 (F3) - population and context, Facet 4 (F4) - SDLC activity, and Facet 5 (F5) - agent / tool profile. The F1 distribution is dominated by Evaluation Research (71.7%) and Validation Research (22.3%); the F5 paradigm distribution is dominated by Pro-code tools (84%), with Low-code and No-code sub-corpora thin (5% and 11% respectively).

2. **A five-mode Human–AI Interaction Mode Taxonomy** (RQ2) that orders along a delegation-depth axis from assistive to autonomous engagement: It identifies five distinct interaction modes: 1) Inline Completion, 2) Conversational Prompting, 3) Visual/Declarative Composition, 4) Review & Validation, and 5) Delegated Task Execution. By identifying empirically recurring modes of human–AI interaction that can be read against SDLC activities, tool profiles, and populations, provides a common vocabulary for downstream demand-side study design. The taxonomy was inductively derived from the canonical codes extracted from a corpus of 290 papers; theoretical saturation was achieved at the mode layer within the final 10% of the sample. To facilitate application, the taxonomy includes operational definitions, distinguishing criteria, and paraphrased exemplars
3. **A capability–SDLC gap matrix** (RQ3) that harmonizes supply-side capabilities from the three reference surveys into 19 unified IDs and cross-tabulating them against nine SDLC activities. By quantifying, for every capability that supply-side surveys emphasize whether and where empirical demand-side evidence has accumulated, making the capability-versus-evidence asymmetrically visible at a granular level rather than at a survey level.

To address the practical implications of RQ3, the study proposes a pre-registered research agenda of seven Wieringa-typed [32] Calls for downstream studies. The agenda has two parts. Six corpus-internal Calls target the 42 flagged supply-versus-demand gaps: Calls 1–5 (Evaluation or Validation Research) cover the 13 highest-priority gaps at priority ≥ 1.00 , and Call 6 (integrative Evaluation Research) bundles a cluster of 10 moderate-priority gaps at priority = 0.50. A seventh Call extends the agenda beyond the corpus-internal gap matrix as a downstream taxonomy construction and validation study addressing the Pro-code-dominance transferability boundary. Every priority ≥ 0.50 flagged gap maps to at least one Call, so the agenda names which empirical studies to run next, for which populations, and how the taxonomy should be extended beyond the Pro-code sub-corpus. Specifically, the seven Calls are:

- Call 1: AI code completion at early-lifecycle activities;
- Call 2: AI code translation beyond the coding boundary;
- Call 3: Agentic self-reflection outside Coding;
- Call 4: Multi-agent orchestration at under-studied SDLC intersections;
- Call 5: AI vulnerability detection beyond Coding;
- Call 6: Knowledge-management capabilities at non-native SDLC activities; and
- Call 7: A taxonomy of AI-agent tool architectures beyond the Pro-code-dominant corpus.

Each Call carries a research-question template and a population-fit note (Appendix C.5). Organizations, tool builders, educators, and researchers can adopt the agenda as a pre-registered prioritization rather than editorial commentary.

1.4 Paper Organization

Section 2 positions the study against the reference surveys and prior taxonomies of developer–AI interaction. Section 3 describes the study design: search, screening, classification, extraction, open coding, and the three analysis tracks. Section 4 reports findings by RQ. Section 5 discusses practical implications and presents the research agenda. Section 6 audits trustworthiness along the four qualitative-research criteria. Section 7 concludes. An appendix packages the codebook, per-mode operational definitions and exemplars, per-gap rationale, the comparison matrix, and the eight transferability boundaries.

2 BACKGROUND AND RELATED WORK

2.1 The supply-side reference surveys

Three recent surveys establish the foundation for this research. Wang et al. [29] review AI agentic programming, classifying systems across behavioral dimensions like tool augmentation and adaptability. A second survey by Wang et al. [30] examines software engineering agents through a perception-memory-action framework, while Otoum & Elkhaili [20] provide a methods-oriented review structured around lifecycle integration and evaluation. While all three acknowledge adoption as a critical open question and identify core capabilities, such as code completion, program repair, and multi-agent collaboration, none establishes a demand-side taxonomy or cross-tabulates these capabilities against empirical usage. This study addresses these limitations by synthesizing their disparate capability vocabularies into 19 unified identifiers (§3.5, Table 4) and cross-tabulating them against nine SDLC activities (§4.3).

2.2 Prior developer-AI interaction taxonomies

Two pieces of prior work directly address the developer-AI interaction-pattern question posed by RQ2. Treude & Gerosa [27] is a Solution Proposal paper that presented a conceptual taxonomy of author-identified interaction types rather than primary-source evidence of human interaction with AI-SE tools; Kumar et al. [17] is a qualitative observational study at a single company, Microsoft, whose empirical-usage character is in scope but whose single-site sampling fell below the corpus-inclusion rule. Both are therefore external methodology references rather than corpus papers; this is a standard positioning for mapping-study prior-work sections.

Treude & Gerosa [27] propose a conceptual taxonomy of eleven developer-AI interaction types at the user-interface mechanism level (auto-complete suggestions, command-driven actions, conversational assistance, contextual recommendations, selection-based enhancements, explicit UI actions, comment-guided prompts, event-based triggers, shortcut-activated commands, file-aware suggestions and automated API responses). The taxonomy is explicitly pre-empirical, not derived from a literature corpus and not cross-referenced against a supply-side capability inventory. The taxonomy is also accompanied by a research agenda of eight themes: (a) effectiveness of interaction types, (b) developer trust and adoption, (c) context-aware AI interactions, (d) optimizing for developer control and customization, (e) reducing cognitive load, (f) ethics and bias, (g) privacy and data protection, and (h) hallucination and damage control.

Of those eight, four are operationalized within the present agenda: theme (a) by Calls 1–6 at the capability $\times$ SDLC-activity level, theme (b) by Call 5’s security-alert triage study, theme (d) by the delegation-depth axis of §4.2 that quantifies developer-versus-agent control, and theme (h) by Call 3’s agentic-self-reflection study. The remaining four, i.e. (c) context-aware AI interaction, (e) cognitive load, (f) ethics and bias, (g) privacy, are deliberately scoped out of the demand-side mapping: (c) is a tool-design research direction that belongs to the design-side literature, and (e), (f), and (g) are cross-cutting human-factors covariates that the present study pre-registers in §5.3 as universal data-collection requirements rather than as standalone Calls.

Kumar et al. [16] contribute a qualitative observational study of 19 professional developers using Cursor Agent to resolve 33 open issues in repositories they had previously contributed to. The study is one of the first in-the-wild empirical accounts of developer-agent collaboration in an integrated development environment. The study organizes its findings around three questions: Kumar-RQ1 (collaboration patterns) - how developers collaborate with a SWE agent, Kumar-RQ2 (barriers) - what barriers developers face in that collaboration, and Kumar-RQ3 (success factors) - what factors predict success.

Their empirical findings inform the present research agenda in two concrete ways. First, Kumar-RQ1 identifies two distinct delegation strategies that developers adopt when working with a SWE agent: 1) a one-shot strategy in which the developer hands the task to the agent as a single prompt, and 2) an incremental-resolution strategy in which the developer and agent iterate through sub-tasks; and Kumar-RQ3 reports their differential success rates. Of the 29

eligible issues (the 33 attempted minus 4 abandoned for reasons external to the agent), 83% of issues attempted under the incremental-resolution strategy reached satisfactory resolution, compared with 38% of issues attempted under the one-shot strategy; and 64% of the 22 issues where developers supplied expert insights reached satisfactory resolution, compared with 29% of the 7 issues where no expert insights were supplied. These observations give Mode 5 Delegated Task Execution (§4.2) an empirical interior structure at prompt-turn granularity that the present SMS's corpus-level synthesis could not produce by itself, and Mode 5 therefore carries Kumar et al.'s strategy distinction forward as a sub-mode refinement. Second, the Kumar-RQ2 barrier findings: lack of tacit knowledge, unsolicited agent actions, sycophancy, and overconfidence, are adopted as pre-registered measurement targets in Calls 3 and 4 of the present research agenda (§5.3), so that any downstream study executing those Calls carries forward Kumar et al.'s documented failure modes as explicit falsifiability conditions. Kumar et al.'s two supporting taxonomies: the Participant Activity Taxonomy and the Chat Trajectory Taxonomy (Tables III and IV in the source paper), sit at prompt-turn granularity within Mode 5 and complement the delegation-depth axis.

The present study complements both prior works along three axes: (i) it synthesizes 640 primary empirical studies at corpus scale rather than cataloguing UI modalities or observing a single-site sample; (ii) it orders interaction modes along a delegation-depth axis rather than a UI-modality axis or a prompt-turn axis; and (iii) it cross-references the emergent taxonomy against a supply-side capability matrix so that mode-layer findings can be read against the coverage of named capabilities. Treude & Gerosa's Section IV conclusion explicitly calls for "systematic evaluation of existing tools to analyze the frequency and usage of each interaction type"; the present SMS partially executes that call at the capability $\times$ SDLC-cell level, even though it adopts a different taxonomic axis.

2.3 Adjacent work on platform paradigms

Two adjacent surveys bound the platform-paradigm part of the scope. Ajimati et al [1] systematically review low-code and no-code development adoption and name citizen-developer contexts explicitly. Liu et al. [19] survey large language model-based agents for software engineering, overlapping with the two Wang surveys on the supply side but with a broader focus on LLM capabilities outside the agentic-programming frame. Both are used here as snowballing seeds (§3.2) in addition to the three primary reference surveys.

2.4 Methodological anchors

The study uses four methodological anchors, each cited at the point where a decision is made. Petersen et al. [23, 24] govern the systematic mapping study (SMS) design: the five-facet classification scheme, the PRISMA attrition flow, and the visualization conventions in §4.1. Cruzes & Dybå [7, 8] govern the five-step thematic synthesis that produces the RQ2 taxonomy, together with the separation of extraction (stable descriptive coding) from synthesis (derived higher-order themes). Wieringa et al. [32] govern the F1 contribution-type vocabulary and the Evaluation / Validation Research distinction used in the research agenda (§5.3). Ali & Petersen [4] govern the backward snowballing procedure. The trustworthiness audit in §6 applies Lincoln & Guba's [18] four qualitative-research criteria — credibility, confirmability, dependability, and transferability — as adapted to software engineering by Cruzes & Dybå [7].

2.5 Positioning

This SMS is therefore positioned as the first demand-side complement to a supply-side-saturated subfield. It does not compete with the three reference surveys; it maps the human-side usage distribution they do not aim to characterize. Table 4 (Appendix C) renders the four objectives $\times$ three reference surveys novelty-comparison matrix and documents zero overlap at any cell.

3 METHOD

3.1 Study design

This study is a systematic mapping study [23, 24]. SMS is appropriate here because AI-SE is an emerging subfield in which scoping, classification, and trend identification are more tractable than pooled effect estimation. Execution proceeded through seven phases - Phase 0: protocol scaffolding, Phase 1: protocol finalization and capability harmonization, Phase 2: search and screening, Phase 3: full-text extraction with concurrent open coding, Phase 4: code consolidation and theme translation, Phase 5: frequency analysis and cross-tabulation, Phase 6: trustworthiness assessment, and Phase 7: manuscript drafting. Every phase produces a tagged artefact set and a commit checkpoint; every borderline classification decision is logged in an append-only decision register with the rule applied and the rationale.

3.2 Protocol scaffolding and finalization

The study protocol was developed in two distinct phases prior to the formal execution of the search. Phase 0 (Scaffolding) involved the commitment of the initial pre-registered protocol to version control, encompassing the research questions, a Systematic Mapping Study (SMS) design outline, a preliminary codebook, an extraction form template, and six Scopus search-string archetypes.

Phase 1 (Finalization) yielded four primary protocol artifacts that served as the foundational framework for subsequent stages:

- **Five-Facet Codebook:** Operationalizes the F1–F5 vocabularies through formal definitions, positive/negative exemplars, and boundary-case rules (codebook.md).
- **Harmonized Capability Vocabulary:** A consolidated 19-capability taxonomy that collapses near-synonyms identified across the three primary reference surveys (capability_list.csv).
- **Extraction Schema:** Defines the per-paper data fields, sample characterization metrics, and capability-annotation formats (extraction_schema.md).
- **Inclusion and Exclusion Criteria (IC/EC):** Establishes the mechanical filters and manual screening heuristics (inclusion_exclusion.md; see Appendix A.2).

The operational definitions of the IC/EC and codebook boundary cases were subsequently validated through a screening pilot conducted on a convenience sample. Refinements emerging from this pilot were documented in a formal report (pilot_report.md) and integrated into the final criteria. Post-Phase 1, the protocol artifacts remained immutable; any subsequent deviations were recorded as formal entries within a centralized decision register.

3.3 Search strategy

The systematic search was conducted through a multi-stage process involving automated database queries and manual citation tracking. To ensure comprehensive coverage, six Scopus search strings were executed in parallel, targeting distinct domains: code-completion tools, conversational assistants, agentic Software Engineering (SE) tools, AI-augmented low-code/no-code platforms, visual/declarative programming, and citizen-developer contexts. The Scopus result set was complemented by a dedicated ACM Digital Library run using ACM-DL query syntax, and supplemented by backward snowballing [4] from the five seed surveys: the three reference surveys [20, 29, 30] plus two topic-adjacent papers [1, 19]. Duplicate overlap between ACM-DL and Scopus was removed at deduplication. The full search strings, per-string hit counts, and replacement decision are reproduced in Appendix A.1.

3.4 Inclusion and exclusion criteria; screening

The review protocol established five inclusion (IC) and six exclusion (EC) criteria prior to screening (Appendix A.2). To be included, papers required: (IC1) publication between 2022–2026; (IC2) empirical evidence of human interaction with AI software engineering agents, AI coding tools, or AI-enhanced low-code platforms; (IC3) peer-reviewed status; (IC4) English language; and (IC5) a minimum length of four pages. Conversely, papers were excluded if they were: (EC1) solution proposals lacking user evidence; (EC2) benchmark-only studies without human participants; (EC3) outside the software engineering domain; (EC4) secondary studies (e.g., literature reviews); (EC5) short papers, posters, or extended abstracts; or (EC6) inaccessible for full-text retrieval within two weeks.

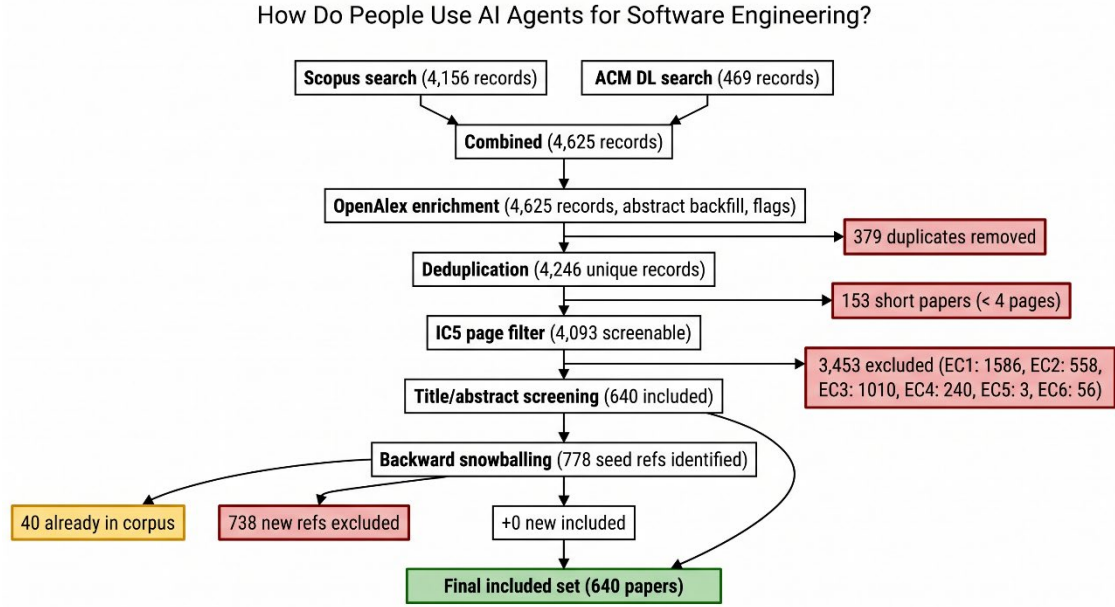


Figure 1: Illustration of the corresponding PRISMA screening flow

Figure 1 illustrates the corresponding PRISMA screening flow. The initial search yielded 4,625 records (4,156 Scopus; 469 ACM DL). After removing 379 duplicates and filtering out 153 short papers (IC5), 4,093 records advanced to title and abstract screening, resulting in 3,453 exclusions. Backward snowballing from five seed surveys yielded no new additions, as all relevant references were already captured. The final included corpus comprises 640 papers. A full reconciliation of execution-time labels against the formal criteria is provided in Appendix A.3.

3.5 Classification Scheme

Consistent with established Systematic Mapping Study (SMS) guidelines [23], our classification scheme provides the analytical framework to address all research questions. We categorized each included paper across five core facets (F1–F5). Two additional dimensions specified in the protocol, Interaction Mode and Capability Category, were treated as derived deliverables for RQ2 (§4.2) and RQ3 (§4.3) rather than initial screening facets. This distinction maintains a rigorous boundary between data extraction and synthesis [7].

- **F1 — Contribution type (RQ1).** The paper’s primary research contribution in the six-class Wieringa et al. [32] vocabulary: Evaluation Research, Validation Research, Solution Proposal, Philosophical, Opinion, and Personal Experience. F1 is single-valued per paper, coded at title and abstract level during screening, and revised at full-text read-through when warranted.

- **F2 — Research methodology** (RQ1). The empirical method used to collect and analyze human-interaction evidence. Seven canonical values: 1) Experiment, 2) Case Study, 3) Mixed, Survey, 4) Mining Study, 5) Interview, and 7) Field Study, are defined in the pre-registered codebook (Appendix A.4). F2 is single-valued; when a study combines two or more methods (for example survey plus interview), the paper is coded Mixed and the component methods are recorded in the extraction-form notes column.
- **F3 — Population and context** (RQ1, RQ2). This facet characterizes the study participants and their operational environment by intersecting a population axis (*Professional Software Engineer, Student, Citizen Developer, OSS Contributor, or Mixed*) with a context axis (*Industry, Education, OSS, Lab, or Mixed*). Studies lacking a distinct human cohort, primarily automated benchmark evaluations, were coded as *Not Applicable* (N/A). Boundary-case papers involving non-developer end-users (e.g., older adults, creative writers, accessibility testers) were mapped to the closest applicable category and explicitly logged in the decision register as accepted edge cases.
- **F4 — SDLC activity** (RQ1, RQ3). This facet identifies the specific stage of the SDLC where the AI tool is applied, utilizing a nine-category taxonomy: Requirements, Design, Coding, Testing, Code Review, Debugging, CI/CD, Documentation, and Project Management. Because empirical studies frequently evaluate tools across multiple tasks (e.g., both coding and debugging), F4 is coded as a multi-valued facet allowing concurrent assignments. To maintain analytical precision, all visualizations derived from this facet explicitly declare their unit of analysis (i.e., distinct paper versus paper-activity instances) in the corresponding figure captions.
- **F5 — Agent / tool profile** (RQ1, RQ2). This composite facet characterizes the evaluated systems through two distinct sub-dimensions. The Development Paradigm (*Pro-code, Low-code, or No-code*) is a single-valued classification defining the tool's overarching category. The Interaction Modality (*Autocomplete, Conversational, IDE-Integrated, or Autonomous*) categorizes how AI capabilities are surfaced to the user. Because contemporary tools frequently employ hybrid interfaces, such as combining autocomplete with conversational agents, the modality sub-facet is multi-valued. Enforcing a mutually exclusive categorization would artificially suppress the empirical signal regarding these multi-modal interactions.

The F1 vocabulary adopts Wieringa’s established classification scheme [32]. Facets F2–F5 employ study-specific controlled vocabularies detailed in Appendix A.4, providing rigorous operational definitions, positive and negative exemplars, and heuristics for boundary cases.

A critical conceptual distinction underpins the analysis in §4.2 and beyond: F5 modality is treated as an intrinsic system attribute, whereas the RQ2 interaction mode (§4.2) is an inductively derived pattern of human-AI behavior. To enforce this separation, Appendix B.1 formalizes a tripartite taxonomy distinguishing capability (*what the agent can do*), modality (*how the tool surfaces the capability*), and interaction mode (*how the human systematically engages with the tool*).

3.6 Extraction and open coding

Data extraction utilized a structured schema (Appendix A.5) to capture bibliographic metadata, F1–F5 facet classifications, sample characteristics, verbatim human–AI interaction passages, and capability annotations. To standardize these annotations, we developed a harmonized 19-capability vocabulary by merging and deduplicating capability lists from the three core reference surveys (Appendix A.6).

Extraction operated under two distinct modes depending on full-text availability:

- **Mode A (Full-Text Extraction):** Applied to 290 papers (45.3% of the corpus). This subset includes 258 papers successfully recovered via a four-tier retrieval cascade (Unpaywall, Semantic Scholar, arXiv, and manual search). First-pass open coding [7] of the verbatim passages in these papers generated 1,016 in-vivo and descriptive codes. Because of this qualitative depth, Mode A papers contributed to all research questions (RQ1–RQ3), including the inductive synthesis required for RQ2.
- **Mode B (Abstract-Only Extraction):** Applied to the remaining 350 papers (54.7%) where full texts were inaccessible. Because abstracts lack the depth required for passage-level qualitative analysis, Mode B papers contributed exclusively to the RQ1 facet classifications and RQ3 capability annotations.

To ensure analytical precision, the applicable denominator (Mode A vs. full corpus) is explicitly disclosed wherever relevant in the results. This Mode A/B split constitutes a key transferability boundary for the study, which is further detailed in §6 and Appendix C.2.

3.7 Analysis

The analysis proceeded along three distinct methodological tracks aligned with our research questions:

- **RQ1: Descriptive Frequency Analysis.** Distributions across facets F1–F5 were computed for the full 640-paper corpus. In accordance with established Systematic Mapping Study (SMS) conventions [23, 24], these distributions are visualized using a bubble plot, bar charts, and heatmaps. Detailed figure notes (Appendix B.2) explicitly define the counting units and the handling of multi-valued cells to ensure interpretability.
- **RQ2: Inductive Thematic Synthesis.** We employed a five-step thematic synthesis workflow [7, 8], executed across three operational passes:
 - Pass 1: Generated 1,016 open codes during initial data extraction (src: artifacts/extraction/open_codes_pass1.csv).
 - Pass 2: Clustered these into 707 canonical labels (src: artifacts/synthesis/consolidated_codes.csv) using sentence-transformer embeddings (cosine-similarity threshold of 0.35). All cluster modifications (merges, splits, and renames) required manual rater approval and were logged in the decision register.
 - Pass 3: Inductively classified each canonical label into five emergent interaction modes (or a "Residuals" category) distributed along a pre-committed delegation-depth axis.

During Step 4 of the workflow, an initial frame of eight provisional modes was condensed to the final five: Paired Collaboration was merged into Conversational Prompting; Scaffolded Guidance was distributed across Conversational Prompting and Delegated Task Execution; and Human-In-The-Loop (HITL) Delegation and Autonomous Orchestration were combined into Delegated Task Execution.

Theoretical saturation was verified using the strict stopping rule of "zero new modes in the final 10% of papers" [7, 8]. Processing order logs confirm that the final 29 of the 290 full-text papers yielded no new modes. Finally, to ensure verbatim originality, all mode-layer exemplars underwent automated offline paraphrase-linting against source passages using a 15-word substring overlap heuristic.

- **RQ3: supply-versus-demand cross-tabulation.** We cross-tabulated the 19 harmonized capabilities against the nine F4 Software Development Lifecycle (SDLC) activities to construct a 19×9 (171-cell) matrices. Cells represent the count of distinct papers that possessed both a capability annotation and an F4 value. We employed a strict gap definition: a cell is flagged as an empirical "gap" if its evidence count is strictly positive but at or below the 25th percentile (P25) of the non-zero cell distribution. Cells with zero evidence are classified distinctly as "empty" rather than "gaps," delineating the difference between under-

studied and untouched intersections. For this corpus, the non-zero P25 is 2.0 (src: artifacts/analysis/prioritised_gaps.md header block); thus, gaps correspond to cells with counts of 1 or 2. Sensitivity analyses at the 33rd percentile and robustness checks spanning P20–P40 are detailed in Appendix C.3. The 42 identified gaps were each mapped to an explanatory hypothesis category (adoption_lag, design_gap, or organizational_readiness_barrier). To preserve the methodological boundary between data extraction and synthesis [7], these qualitative hypothesis labels were recorded exclusively in the decision register rather than appended as columns directly in the data matrix.

4 RESULTS

4.1 RQ1 — Evidence landscape

Corpus volume and growth. The final corpus comprises 640 studies published between 2022 and 2026, with a pronounced recency skew: 584 of 640 papers (91.3%) appear from 2024 onward. The 2026 count is partial because the search cut-off precedes year end. This distribution is consistent with the post-release inflection in generative-AI tooling, particularly following ChatGPT and GitHub Copilot, and situates the SMS within a rapidly emerging research area. Regarding venue type, 456 papers (71.2%) are conference publications and 184 (28.8%) are journal articles. Disaggregated counts by year, F1 class, and F2 method are reported in Table 1.

Table 1: Corpus descriptives (N = 640)

Year	Count	%	F1 Class	Count	%	F2 Method	Count	%
2022	8	1.2%	Evaluation Research	459	71.7%	Experiment	249	38.9%
2023	48	7.5%	Validation Research	143	22.3%	Mixed	139	21.7%
2024	226	35.3%	Solution Proposal	29	4.5%	Case Study	92	14.4%
2025	288	45.0%	Personal Experience	7	1.1%	Mining Study	63	9.8%
2026	70	10.9%	Opinion	2	0.3%	Survey	63	9.8%
						Interview	20	3.1%
						Field Study	14	2.2%

F1 — empirical evaluation dominates. Evaluation Research constitutes the majority of the corpus (459 papers, 71.7%), followed by Validation Research (143, 22.3%). By contrast, Solution Proposals (29, 4.5%), Personal Experience reports (7), and Opinion papers (2) are sparsely represented (source: artifacts/extraction/extraction_matrix.csv). The corpus is therefore overwhelmingly oriented toward empirical investigation of AI-enabled software engineering (AI-SE) in context, consistent with the study’s demand-side framing and the explicit exclusion of non-empirical work under ECs 1 and 2..

F2 — controlled experiments dominate; qualitative methods are thin. Experimental designs account for the largest share (249 papers, 38.9%), followed by Mixed methods (139, 21.7%), Case Studies (92, 14.4%), Mining Studies (63, 9.8%), and Surveys (63, 9.8%), collectively covering 606 of 640 papers (94.7%). In contrast, Interview-based studies (20, 3.1%) and Field Studies (14, 2.2%) together comprise only 34 papers (Figure 2). This relative scarcity of in situ, first-person qualitative inquiry provides an independent rationale for subsequent research employing such methodologies.

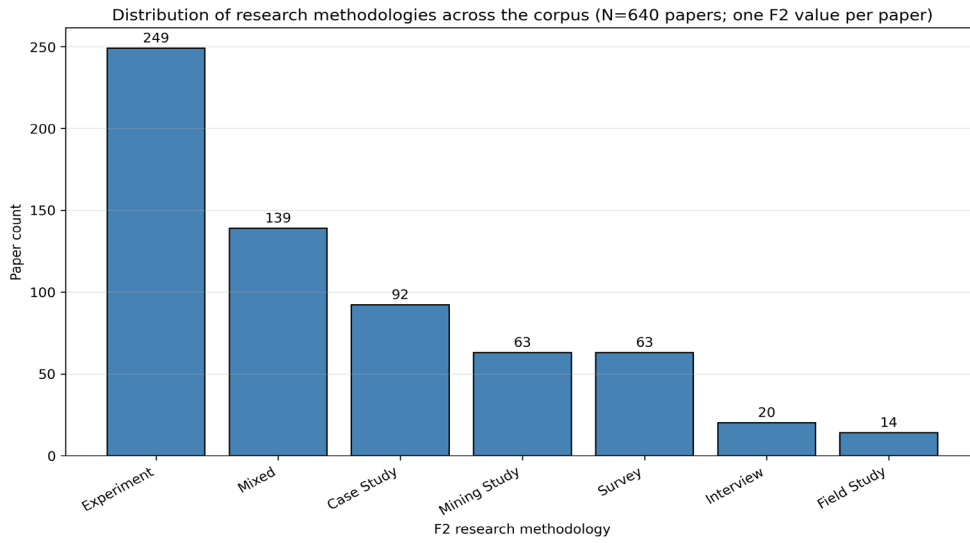


Figure 2: F2 research-methodology distribution for the 640-paper corpus.

F3 — Population coverage is incomplete. Of the 640 studies, 392 (61.3%) report a canonical F3 population value, while 248 (38.7%) are coded as missing (NaN), predominantly corresponding to benchmark-oriented evaluations that do not describe human participants. Within the 392-paper sub-corpus, the distributions across Student (133; 20.8% of the full corpus), Professional SWE (128; 20.0%), and Mixed populations (109; 17.0%) are broadly comparable in magnitude. In contrast, Citizen Developer (19; 3.0%) and OSS Contributor (3; 0.5%) are markedly underrepresented. Figure 3 presents the year-by-SDLC bubble distribution faceted by F3 population, where the panels corresponding to Citizen Developer and OSS Contributor are visibly sparse.

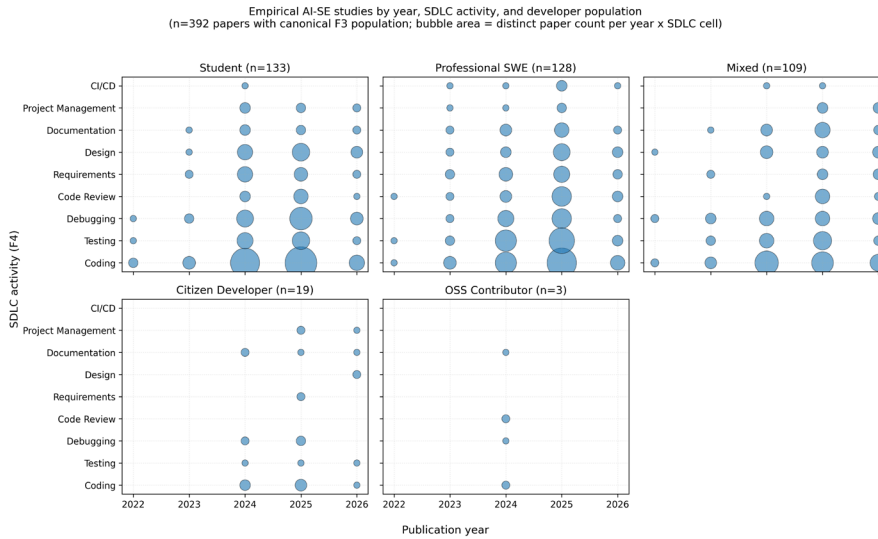


Figure 3: Year × SDLC activity × developer population bubble plot (n = 392 of 640 papers with canonical F3 population).

F4 — SDLC coverage is code-centric. The 640 studies yield 1,083 paper–SDLC activity bindings, distributed unevenly across the nine-activity taxonomy. Activities centered on Coding, Testing, and Debugging account for most bindings, whereas Requirements Engineering, Project Management, and CI/CD are comparatively underrepresented. Figure 4 presents the SDLC-by-year heatmap. This structural asymmetry in activity coverage provides an antecedent condition for the gap patterns identified in RQ3 (§4.3).

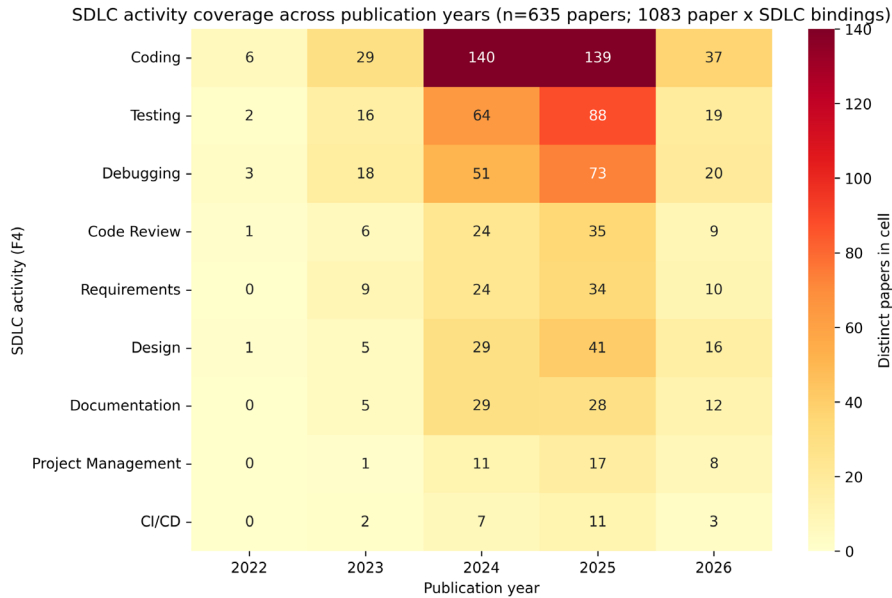


Figure 4: SDLC × year evidence heatmap. Each cell is the distinct-paper count for the given SDLC activity and publication year. Coding, Testing, and Debugging dominate; Requirements, Project Management, and CI/CD are thin across every year.

F5 — Pro-code paradigm predominates. Of the 640 studies, 538 (84.1%) adopt a pro-code paradigm, compared with 72 (11.3%) no-code and 30 (4.7%) low-code studies (Figure 5). The low-code and no-code strata are comparatively data-sparse; consequently, paradigm-specific inferences reported in §4.3 should be interpreted with appropriate transferability caveats (see §6 and Appendix C.2). Note: Total n=639 of 640 corpus papers in Figure 5, the single excluded paper having a NaN modality and therefore no plottable stack segment

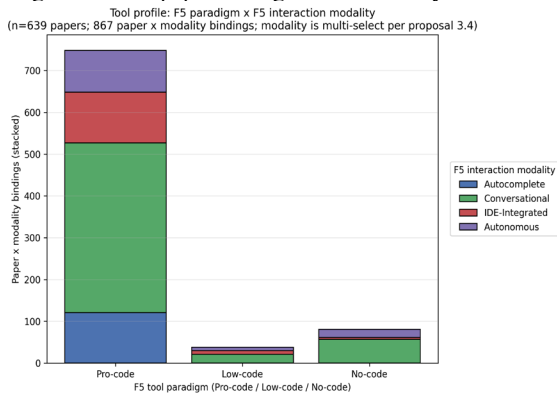


Figure 5 F5 tool profile. Paradigm (Pro-code, Low-code, No-code) on the horizontal axis; interaction-modality stacks (Autocomplete, Conversational, IDE-Integrated, Autonomous) within each paradigm bar.

4.2 RQ2 — Human–AI Interaction Mode Taxonomy

The Delegation-Depth Axis. We define delegation depth as the relative distribution of task planning, execution, and verification responsibilities between the human and the AI. This spectrum ranges from Mode 1, where the human strictly retains all three functions, to Mode 5, where the human cedes planning and execution, retaining only the final verification checkpoint. While this overarching axis was established *a priori* in our protocol, the specific interaction modes arrayed along it were derived inductively from the empirical data.

Five modes arrayed along the axis. The inductive synthesis of 707 canonical labels yielded a comprehensive five-mode taxonomy, summarized in Table 2 (*source: artifacts/synthesis/interaction_taxonomy.md*). Full operational definitions, inclusion/exclusion boundary criteria, and paraphrased exemplars are detailed in Appendix B.3.

Table 2: Five-mode interaction taxonomy summary

#	Mode	Canonical labels	Share (%)	Saturation
1	Inline Completion	4	0.6%	Stable; corpus-bias caveat (§4.2 & §6)
2	Conversational Prompting	69	9.8%	Stable
3	Visual / Declarative Composition	9	1.3%	Stable; paradigm-skew caveat
4	Review & Validation	82	11.6%	Stable
5	Delegated Task Execution	75	10.6%	Stable
r	Residuals (outcome / constraint / meta / affordance)	468	66.2%	Sub-structured (§4.2, Appendix B.3)

Mode 1 thinness reflects corpus visibility rather than usage sparsity. The four canonical Mode 1 labels fall well below the 15-label re-open trigger specified in the pre-registered synthesis workflow. We therefore interpret Mode 1 thinness as a property of the published and retained corpus, not as evidence of sparse practitioner use. Inline completion has been in production since 2021 and is no longer treated as a publication-worthy novelty in many venues. In addition, many short Mode-1-style usage reports were excluded under Wieringa’s non-empirical categories, namely EC1 Solution Proposal or EC2 Validation Research, or under EC5 as short papers or extended abstracts. Accordingly, all claims about Mode 1 prevalence are reported with explicit corpus-size caveats in §6 and Appendix C.2.

Residuals exhibit a coherent sub-structure. The 468 Residual labels partition into four evidence-about-interaction sub-layers: outcome labels (192, 41%), constraint labels (119, 25%), meta labels (101, 22%), and affordance labels (56, 12%). This partition suggests that many empirical AI-SE papers report evidence surrounding interaction, including outcomes of use, constraints and barriers, adoption drivers, and affordances, rather than reporting mode-defining interaction episodes themselves. We therefore report the five-mode partition and the Residual sub-structure jointly, while treating them as distinct empirical signals.

Mode × SDLC variation shows universal Coding dominance. In the passage-bearing subsample, Coding is the leading SDLC activity for every mode row: Mode 1, 80%; Mode 2, 60%; Mode 3, 82%; Mode 4, 53%; Mode 5, 58%; and Residuals, 61% (*source: artifacts/analysis/rq2_variation/mode_x_sdlc.md*). Mode 5 has the strongest CI/CD footprint, with 11 papers, or 9.6% of its passage-bearing subsample, consistent with the delegation-axis expectation that autonomous modes extend into operational pipelines. Mode 4 has the highest Code Review share, at 25.5%, consistent with its evaluative stance. Full row-percent tables are reported in Appendix B.4.

Mode × tool profile - Pro-code dominates across the delegation axis, with Mode 3 as the Low-code exception. Among the 290 passage-bearing papers, each carrying an F5 paradigm value, 253 are Pro-code (87.2%), 26 are No-code (9.0%), and 11 are Low-code (3.8%) (src: artifacts/extraction/extraction_matrix.csv joined against the 290 passage-bearing paper_ids in artifacts/extraction/open_codes_pass1.csv). No-code papers appear in Modes 2, 3, 4, and 5; Low-code papers appear in the same four modes. Mode 3 (Visual / Declarative Composition) uniquely carries a substantial Low-code share — 18.2% of its 11-paper subsample, versus at most 3.5% in any other mode — consistent

with its visual and declarative construct. Mode 3's No-code share (9.1%) is comparable to Modes 2 (11.7%), 4 (10.4%), and 5 (8.8%), so No-code evidence is distributed across several modes rather than concentrated in one.

Mode $\times$ population and context reveal thin Citizen-Developer and OSS cells. Across the passage-bearing subsample, the Professional SWE / Industry cell contains 54 papers (21.2%), and the Student / Education cell contains 43 papers (16.9%). Mixed / Lab accounts for 27 papers (10.6%), and Professional SWE / Lab accounts for 13 papers (5.1%). By contrast, Citizen-Developer and OSS-Contributor populations together account for only 7 of 255 papers (2.7%). These cells are therefore too thin to support mode-level prevalence claims. The industry-versus-education comparison identified in the proposal as a gap in prior literature remains underpowered in this corpus and is carried forward into the research agenda in §5.3.

Mode-layer saturation was reached. No new modes emerged in the final 29 of 290 passage-bearing papers in processing order. The five-mode partition is therefore stable within the corpus and serves as the RQ2 dependability anchor, as discussed in §6 and Appendix C.3. Canonical-layer saturation was not reached: new canonical labels continued to appear late in the process. This result is expected because the canonical layer is finer-grained than the mode partition.

4.3 RQ3 — Capability–evidence gap matrix

Research Question 3 evaluates the alignment between the 19 capabilities emphasized in the supply-side reference surveys and the available demand-side empirical evidence, specifically identifying areas where empirical support is sparse. To systematically map this landscape, we constructed a 19×9 cross-tabulation matrix (171 distinct intersections) plotting the 19 harmonized capabilities (rows) against the nine SDLC activities (columns). Each cell quantifies the distinct number of papers documenting a specific capability applied within a given SDLC phase (e.g., CAP_CODECOMP $\times$ Requirements represents studies evaluating AI code completion during requirements elicitation).

The underlying dataset comprises 776 capability annotations extracted from 384 distinct papers (artifacts/extraction/capability_annotations.csv). Of this subset, 383 papers possessed complete F4 SDLC metadata and successfully populated the matrix; the remaining annotations lacking an explicit F4 classification were excluded from this final tabulation.

Matrix Partitioning and Gap Definition. Applying the strict, single-rule gap definition (§3.7), the 171 matrix cells partition into three distinct regions: 117 adequately populated cells (68.4%), 42 gap cells (24.6%), and 12 empty cells (7.0%) (source: artifacts/analysis/rq3_gap_matrix/gap_matrix.csv). We define a gap as an intersection with strictly positive empirical evidence that falls at or below the 25th percentile (P25) of the non-zero cell distribution. For this corpus, $P25 = 2.0$; thus, gap cells contain only one or two supporting papers. Cells with zero evidence are classified separately as empty cells to preserve a critical interpretive distinction: gaps represent under-studied intersections, whereas empty cells represent untouched intersections. A sensitivity analysis utilizing a 33rd percentile threshold ($P33 = 3.14$) expands the gap set to 53 cells (artifacts/analysis/rq3_gap_matrix/sensitivity_25_33.md); the complete P20–P40 robustness band is detailed in Appendix C.3.

Evidence Concentration and Sparsity. Empirical evidence is heavily concentrated along the matrix diagonal, mapping core capabilities directly to their native SDLC activities. The three highest-density intersections are CAP_CODEGEN $\times$ Coding (156 papers), CAP_TESTING $\times$ Testing (126 papers), and CAP_DEBUGGING $\times$ Debugging (93 papers). Conversely, off-diagonal evidence is substantially thinner. The combined 54 gap and empty cells are predominantly clustered around early-lifecycle phases (Requirements, Design) and post-deployment operational stages (CI/CD, Project Management). Figure 6 visualizes this distribution, utilizing red borders to demarcate gap cells and diagonal hatching for empty cells. The pronounced visual asymmetry in this matrix directly informs the structural priorities of the research agenda proposed in §5.3.

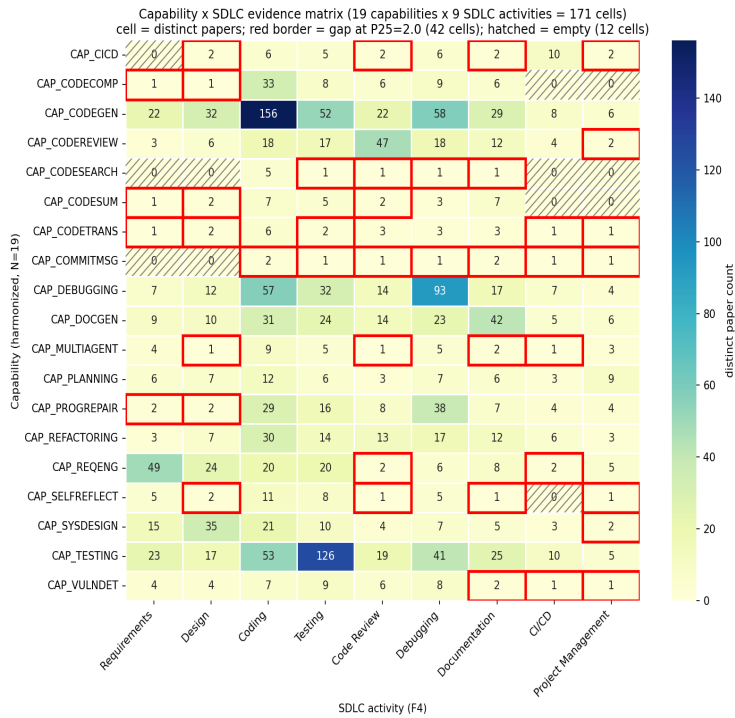


Figure 6: Capability × SDLC evidence matrix (19 capabilities × 9 SDLC activities = 171 cells)

Concentration of Top-Priority Gaps. Table 3 details the 13 top-priority gaps, ranked by a calculated metric: $priority = P25-shortfall \times capability-prevalence$, where prevalence is defined as the number of core reference surveys (1–3) identifying the given capability. Notably, all eight of the highest-priority gaps (score = 1.50) cluster strictly within three core capabilities universally recognized across all three reference surveys: CAP_CODECOMP (intersecting with the Design and Requirements phases), CAP_CODETRANS (intersecting with Requirements, CI/CD, and Project Management), and CAP_SELFREFLECT (intersecting with Code Review, Documentation, and Project Management). The comprehensive 42-row ranked taxonomy is available in Appendix C.4.

Table 3: Top-13 prioritized capability × SDLC gaps (priority ≥ 1.00)

Rank	Mode	Label	SDLC Activity	Evidence	Priority	ERP3 Buket
1	CAP_CODECOMP	Code Completion	Design	1	1.50	design_gap
2	CAP_CODECOMP	Code Completion	Requirements	1	1.50	design_gap
3	CAP_CODETRANS	Code Translation	CI/CD	1	1.50	organizational_readiness_barrier
4	CAP_CODETRANS	Code Translation	Project Management	1	1.50	organizational_readiness_barrier
5	CAP_CODETRANS	Code Translation	Requirements	1	1.50	design_gap
6	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Code Review	1	1.50	adoption_lag
7	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Documentation	1	1.50	adoption_lag
8	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Project Management	1	1.50	adoption_lag
9	CAP_MULTIAGENT	Multi-Agent Collaboration	CI/CD	1	1.00	adoption_lag
10	CAP_MULTIAGENT	Multi-Agent Collaboration	Code Review	1	1.00	adoption_lag
11	CAP_MULTIAGENT	Multi-Agent Collaboration	Design	1	1.00	design_gap
12	CAP_VULNDET	Vulnerability Detection	CI/CD	1	1.00	organizational_readiness_barrier

Rank	Mode	Label	SDLC Activity	Evidence	Priority	ERP3 Buket
13	CAP VULNDET	Vulnerability Detection	Project Management	1	1.00	organizational readiness barrier

Interpretive Categorization of Evidence Gaps. To contextualize the 42 identified gaps, they were qualitatively classified into three interpretive categories (documented in the decision register): organizational readiness barriers (22 gaps; 52.4%), design gaps (14 gaps; 33.3%), and adoption lags (6 gaps; 14.3%). Within this proposed coding framework, organizational readiness emerges as the most frequently assigned category. This distribution suggests a working hypothesis: for many mature capabilities, the scarcity of empirical evidence in non-native SDLC activities may stem more from organizational adoption challenges than from underlying technical limitations.

CI/CD and Early-Lifecycle Asymmetry. Continuous Integration/Continuous Deployment (CI/CD) emerges as the most asymmetric and under-evidenced column in the matrix, hosting five gap cells and four empty cells. Furthermore, early-lifecycle activities (Requirements, Design, and Project Management) collectively account for 18 of the 42 gaps (42.9%). This distribution reflects a persistent concentration of both tool design and empirical research on the coding phase, leaving upstream planning and downstream operational phases comparatively underexplored.

Robustness and Threshold Sensitivity. Sensitivity analysis confirms the stability of the gap matrix. The total gap counts at the 20th (P20), 25th (P25), 33rd (P33), and 40th (P40) percentiles are 42, 42, 53, and 72, respectively (detailed in Appendix C.3). Crucially, the foundational 42-cell set identified at P25 is monotonically preserved across this entire band; no cells exit the gap classification as the threshold increases. Therefore, additions at P33 and P40 serve as advisory expansions rather than threshold-sensitive reversals.

Corpus Sparsity in Paradigm Splits. When applying the gap rule to isolated development paradigms, data sparsity becomes a limiting factor. The Low-code sub-matrix (derived from 30 papers) contains 141 empty cells out of 171, while the No-code sub-matrix (72 papers) contains 123 empty cells. In both subsets, the P25 threshold collapses from the global 2.0 to 1.0. Consequently, due to this boundary limitation, per-paradigm gap counts must be interpreted as directional indicators rather than dispositive findings.

5 DISCUSSION

5.1 Dominant pattern: supply-saturation versus demand-under-evidence

The Supply-Demand Disconnect. A central theme common among all three research questions is the pronounced asymmetry between the saturation of supply-side capabilities and the scarcity of demand-side empirical evidence. While the core reference surveys [20, 29, 30] detail an extensive array of theoretical capabilities (harmonized here into 19 categories), empirical validation remains highly uneven. Of the 171 capability-by-SDLC intersections, nearly a third lack sufficient backing: 42 are classified as under-evidenced gaps, and 12 are entirely empty.

Asymmetry in Human-AI Interaction Modes. This supply-demand asymmetry persists at the Human-AI interaction mode layer (see §4.2). For instance, Mode 5 (Delegated Task Execution), that is identified by reference surveys as the most actively developed technological frontier, accounts for just 10.6% (75 of 707) of the canonical labels in the empirical corpus. Conversely, Mode 1 (Inline Completion), despite ubiquitous mainstream deployment since 2021, represents a mere 0.6% (4 labels); however, this reflects established publication bias against mature tools rather than an absence of practitioner utility. Ultimately, as detailed in the novelty-comparison matrix (Appendix C.1), the reference surveys map the supply side of the AI-SE landscape, whereas this Systematic Mapping Study (SMS) systematically audits the demand side.

Complementarity and Methodological Novelty. Our findings complement, rather than overlap with, Treude and Gerosa’s [27] UI-modality taxonomy. While their framework catalogs interaction modalities at the mechanical user-interface level, this SMS scales interaction modes along an inductively derived delegation-depth axis, which is then cross-referenced against the supply-side capability matrix. To our knowledge, this specific methodological

synthesis—combining a strictly defined statistical cross-tabulation (the 25th-percentile threshold) with a Wieringa-typed prioritization to structure a pre-registered research agenda—has no direct precedent in the AI-SE mapping-study literature.

5.2 Practical implications

The findings of this SMS study yield actionable insights for four distinct stakeholder groups. In each domain, strategic priorities are derived directly from the observed asymmetrical differences between concentrated empirical evidence and identified research gaps.

For **Organizations**. The five Human-AI interaction mode delegation-depth taxonomy (see §4.2) indicates that published organizational AI-SE practice predominantly centers on evaluative and delegated engagement, with Modes 4 and 5 comprising 22.2% (157 of 707) of canonical labels. Industry-context studies corroborate this distribution; for instance, Park [22] demonstrates that the sustained use of AI coding assistants is moderated by perceived usefulness and team-level support, that are factors reflecting transition costs rather than mere feature availability. Similarly, Weisz et al. [31] observe parallel team-level dynamics in industrial deployments. Consequently, the delegation-depth axis serves as a practical self-assessment instrument. Organizations currently at Modes 1 or 2 can map transition trajectories by budgeting for prerequisite capabilities: prompt-engineering fluency for Mode 2, rigorous review protocols for Mode 4, and robust trust-calibration practices for Mode 5. This staged assessment offers greater operational specificity than generic "AI-readiness" scores. Caveat: The underrepresentation of Mode 1 reflects publication bias rather than low practitioner adoption; thus, the corpus-derived distribution should not be interpreted as a strict deployment sequence.

For **Tool Builders**. The 42 flagged gaps distribute across three interpretive categories: organizational readiness barriers (52.4%), design gaps (33.3%), and adoption lags (14.3%). Acknowledging that these classifications serve as pre-registered hypotheses (§4.3), the distribution strongly points toward an adoption-centric frontier. The highest-priority gaps (Table 3), such as code completion and translation in early-lifecycle phases, alongside self-reflection mechanisms in review and project management, highlight critical integration targets for vendors. This is reinforced by Tambon et al. [25], who catalog LLM failure modes in deployment, and Alami et al. [3], who note how LLM-assisted reviews fundamentally alter team accountability. Strategically, further investments in coding-stage translation accuracy will likely yield diminishing returns compared to enhancing CI/CD and project-management integrations. Additionally, the taxonomy informs UX design: optimization targets must shift from interaction friction in Modes 1 and 2, to trust calibration in Mode 4, and checkpoint design in Mode 5. Caveat: The Low-code and No-code sub-corpora are thin (30 and 72 papers, respectively), rendering per-paradigm claims directional rather than definitive.

For **Researchers**. The methodological distribution of the corpus (Experiment 38.9%, Mixed 21.7%, Case Study 14.4%, Interview 3.1%, Field Study 2.2%) reveals a distinct scarcity of qualitative research. Additional controlled experiments are likely to yield diminishing marginal utility, whereas in-situ field studies and practitioner interviews targeting Mode 4 and Mode 5 usage offer high-value empirical signals. This diagnosis aligns with Gao et al. [11], who highlight similar gaps from a supply-side perspective, and Ouyang et al. [21], who argue that LLM non-determinism complicates experimental replication, necessitating triangulated qualitative methods. To operationalize this, the research agenda (§5.3) formulates six corpus-internal Calls (Calls 1–6) and one cross-paradigm taxonomy Call (Call 7). Research groups can adopt these pre-registered priorities to guide future study designs. Caveat: This agenda serves as a structured guide rather than a strict constraint; emergent interview or field-study designs may surface priorities absent from the current gap matrix.

For **Educators**. While student-population studies constitute the largest demographic subset (133 papers), the majority evaluate a single tool (e.g., GitHub Copilot or ChatGPT) within a localized course setting [28]. Critical vulnerability is identified by Bo et al. [5], who document a Mode-5 failure mode where sycophantic LLM outputs actively mislead novice problem-solvers. This underscores that delegated-task assignments necessitate explicit

checkpoint design embedded at the curriculum level. The interaction taxonomy provides a blueprint for a capability-ladder syllabus: establishing Mode 1 fluency in introductory courses, Mode 2 prompting in mid-level programs, and Mode 4 evaluative exercises before introducing Mode 5 delegation tasks. This progression aligns educational frameworks with the empirical trajectory of professional practice rather than transient tool popularity. Caveat: Institutional contexts may require reordering this ladder. Furthermore, comparative analyses between industry and educational deployments remain sparse in the corpus and are marked as an ongoing research priority.

5.3 Research agenda

To operationalize the identified 42 gaps and 12 empty cells, this paper proposes a structured research agenda comprising seven distinct Calls to Action. Calls 1 to 6 are corpus-internal and strictly aligned with Wieringa’s [32] taxonomy (Evaluation and Validation Research) to ensure each Call carries a pre-specified methodological class and operational entry condition. Within this subset, Calls 1–5 formulate concrete primary-study proposals targeting the 13 highest-priority gaps (priority ≥ 1.00). Call 6 operates as an integrative Evaluation Research initiative designed to address a specific cluster of ten moderate-priority gaps (priority = 0.50) centered on knowledge management.

A final directive, Call 7, extends the agenda beyond the internal gap matrix to confront the transferability boundary established by the corpus’s Pro-code dominance. Rather than targeting an isolated matrix gap, Call 7 proposes a downstream taxonomy construction and validation study. Methodologically, it prescribes Evaluation Research grounded in a foundational Multivocal Literature Review [12]. Comprehensive research-question templates and population-fit guidelines for all seven Calls are provided in Appendix C.5.

1. **Call 1: AI Code Completion in Early-Lifecycle Activities** (Evaluation Research; Two priority 1.50 gaps). How do practitioners utilize code-completion tools during requirements elicitation and software design, and how do these usage-patterns diverge from the coding-stage applications that currently dominate the literature?
2. **Call 2: AI Code Translation Beyond the Coding Boundary** (Validation at Requirements; Evaluation at CI/CD and Project Management; Three priority 1.50 gaps). How do legacy-modernization initiatives integrate AI code translation into migration project management and CI/CD pipelines? Furthermore, under what conditions does automated translation reliably preserve specification intent?
3. **Call 3: Agentic Self-Reflection Outside of Coding** (Validation Research; Three priority 1.50 gaps). To what extent does agentic self-reflection yield measurably better outcomes for code-review synthesis, documentation generation, and project-management task decomposition compared to single-pass baselines? Following Kumar et al. [16], downstream studies must pre-register a falsifiable constraint: self-reflection is credited only if its quality advantage persists under user-disagreement conditions, strictly controlling for known sycophancy and overconfidence failure modes.
4. **Call 4: Multi-Agent Orchestration at Under-Studied SDLC Intersections** (Validation Research; Three priority 1.00 gaps). How effectively do multi-agent frameworks perform as collaborative code-review committees, CI/CD pipeline coordinators, and design-stage ideation partners, and which coordination patterns exhibit the greatest robustness? Downstream studies should pre-register two falsifiable metrics from Kumar et al. [16]: the unsolicited-agent-action rate (per prompt-turn) as a measure of coordination quality, and the success differential between one-shot and incremental coordination patterns across these three intersections.
5. **Call 5: AI Vulnerability Detection Beyond Coding** (Evaluation Research; Two priority 1.00 gaps). How is AI-augmented vulnerability detection embedded within CI/CD pipelines and enterprise security management? Additionally, what organizational factors best predict the effective triage of AI-generated security alerts?

6. **Call 6: Knowledge Management in Non-Native Activities** (Evaluation Research; Ten priority 0.50 gaps). Outside the primary coding flow, when and how do practitioners invoke AI-driven code search, summarization, and commit-message generation? Crucially, what marginal value or friction do these capabilities introduce at critical decision points in code review, testing, debugging, documentation, and CI/CD?
7. **Call 7: A taxonomy of AI-agent architectures beyond the Pro-code** (Evaluation Research preceded by a Multivocal Literature Review [12]; no priority-ranked gap, motivated by transferability boundary and by practitioner-reported scenarios outside the corpus). AI-agent tools vary along an architecture-type dimension, independent of the delegation-depth axis of §4.2. The Call asks two linked questions. First, what are the distinct architectural types in practice? Second, for each type, who is the typical user (developer, citizen developer, researcher-builder, or individual end user), what kind of need triggers its use (a business-process need or a software-development-lifecycle need), and does the tool bridge multiple development paradigms (Pro-code, Low-code, No-code)? The five candidate types, each anchored where possible by an A*-ranked corpus exemplar, are: (i) generalist AI authoring IDEs (e.g., GitHub Copilot, Cursor), anchored by Vaithilingam et al. [28] and Park [22]; (ii) horizontal workflow orchestration tools with AI (e.g., n8n, Zapier), anchored by Cheng et al.'s Prompt Sapper [6] and Wu et al.'s AI Chains [33]; (iii) vertical SaaS suites with embedded AI (e.g., Salesforce Einstein, SAP Joule), anchored by Jiang et al.'s SiriusBI [13] and Weisz et al.'s watsonx Code Assistant audit [31]; (iv) agent-construction frameworks and harnesses (e.g., LangGraph, CrewAI), anchored by Epperson et al.'s AGDebugger / AutoGen / Magentic-One study [9]; and (v) deployed personal-AI-assistant platforms (no A*-ranked corpus exemplar yet; this absence is itself part of the empirical signal that motivates the MLR's grey-literature scope). These five are hypotheses to be tested, not pre-committed conclusions; the final count is determined empirically by the MLR and a companion practitioner survey [14, 15].

5.4 Pre-registered survey items for the downstream stage (ERP-3)

To empirically support the seven research calls defined in §5.3, the downstream research phase will administer a targeted practitioner survey [14]. The survey instrument is structured to validate both the architectural typology (Call 7) and the specific capability-by-SDLC gaps (Calls 1–6). The first three items prioritize Call 7, as the concurrent Multivocal Literature Review (MLR) [12] will field-test the architecture typology alongside survey piloting. Items 4–9 directly operationalize Calls 1–6. Comprehensive scoping, cognitive piloting, and survey administration are deferred to the execution phase of ERP-3.

The foundational survey item bank includes:

- **Item 1: Architectural Self-Classification (Call 7).** Practitioners are asked to map their primary tool to a candidate architectural class: generalist AI authoring IDEs (e.g., Cursor, Copilot), horizontal workflow orchestrators (e.g., n8n, Zapier), vertical SaaS with embedded AI (e.g., SAP Joule, Salesforce Agentforce), agent-construction frameworks (e.g., LangGraph, CrewAI, OpenHands [31]), or deployed personal AI assistants. An open-ended option captures unclassified tools.
- **Item 2: Typology Validation (Call 7).** Evaluates the construct validity of the MLR-derived architecture typology by asking practitioners if the proposed categories, criteria, and exemplars accurately reflect their operational reality, while identifying any missing classifications.
- **Item 3: Role-Bridging and Handoff Dynamics (Call 7).** Investigates cross-paradigm workflows by identifying the tool selected when business-process requirements trigger traditional software engineering

tasks. It captures the user's self-identified role (e.g., professional developer, citizen developer, researcher) and the precise point of handoff to engineering teams.

- **Item 4: Delegation-Depth Distribution (Calls 1, 4, 6).** Utilizes a matrix design to map the frequency of AI usage across the five established interaction modes (Inline Completion through Delegated Task Execution) against specific SDLC activities during a typical development sprint.
- **Item 5: Baseline Interaction Frequency.** Quantifies the proportion of coding time dedicated strictly to inline completion (Mode 1) and the frequency of accepting completions without initiating separate conversational turns, providing empirical grounding against the identified publication bias.
- **Item 6: Translation in Non-Native Contexts (Call 2).** Solicits empirical examples and friction points encountered when integrating AI code translation outside of core coding, specifically within upstream requirements, CI/CD pipelines, or legacy migration workflows.
- **Item 7: Agentic Self-Reflection and Deployment Barriers (Call 3).** Audits the deployment of iterative-refinement loops in code review, documentation, and project management. Crucially, it asks non-adopters to categorize their primary implementation blockers as technical, organizational, or trust-related.
- **Item 8: Security Alert Triage in Pipelines (Call 5).** Determines organizational accountability for triaging AI-generated security alerts within CI/CD pipelines and measures the ratio of actioned versus dismissed alerts to assess operational friction.
- **Item 9: Low-Code/No-Code Interaction Mechanics** (Transferability Boundary IV; Calls 1, 6). Asks practitioners in low-code or no-code environments to characterize their primary AI interaction mode, the specific UI mechanics of their toolset, and its corresponding architectural class from Item 1.

6 THREATS TO VALIDITY

Trustworthiness is audited along the four SE-adapted qualitative-research criteria of Cruzes and Dybå [7, 8]. The full audit package, comprising the decision register, the Petersen 2015 §5 SMS reporting checklist, and the eight transferability boundaries, are in Appendix C.2 and in the supplementary material.

Credibility. Every operational rule (facet vocabularies, paraphrase linter, P25 gap rule) is declared before execution and recorded in the protocol. Every RQ2 claim traces through a bidirectional path: mode → canonical label → pass-1 code → passage identifier → paper identifier; and every RQ3 gap has exactly one corresponding hypothesis_for_erp3 row in the decision register recording the ERP3 bucket label.

Confirmability. Every output CSV has a companion .meta.json recording its generating script, inputs, and timestamp. All 42 RQ3 gap cells are paired with a single register row, giving a bidirectional link between the gap matrix and the audit log.

Dependability. The reliability of the RQ2 findings is established by the mode-layer saturation, evidenced by the absence of newly identified modes in the final 29 of the 290 passage-bearing papers. Furthermore, the threshold sensitivity analysis for RQ3 demonstrates that the 42-cell P25 gap set is monotonically preserved across the P20 to P40 percentile band.

Transferability. Findings apply within eight boundary conditions documented in Appendix C.2: (i) the 2022–2026 time window; (ii) the venue set (Scopus + ACM DL + five seed surveys; no grey literature); (iii) English-only coverage; (iv) Pro-code paradigm dominance; (v) partial F3 population coverage; (vi) Mode 1 thin evidence; (vii) the 290 / 350 Mode-A / Mode-B split, which limits RQ2 derivation to the passage-bearing subset; and (viii) the ERP3 hand-off scope, which means the research agenda is a structured pre-registration rather than a validated finding. Low-code and No-code practitioner contexts are known to be under-represented; downstream qualitative work should deliberately over-sample them to counter this boundary.

Single-rater study. This SMS was executed by a single rater, with LLM assistance used mainly as a labeller-proposer at the screening and open-coding stages; the rater approved, renamed, merged, or split every decision, and

every borderline decision is logged in the decision register. An inter-rater reliability pilot is explicitly deferred to the downstream qualitative practitioner-interview study, where practitioner evidence also serves as external validation of the RQ2 taxonomy.

7 CONCLUSION

This systematic mapping study addresses a demand-side question that three recent supply-side surveys [20, 29, 30] left open: how do practitioners—spanning software engineers, students, and citizen developers—actually utilize AI coding assistants, software engineering agents, and AI-augmented low-code/no-code platforms? Analyzing a corpus of 640 primary empirical papers published between 2022 and 2026, this study delivers three core contributions:

- **RQ1 (Evidence Landscape):** A comprehensive five-facet mapping of the current empirical literature.
- **RQ2 (Interaction Taxonomy):** A five-mode taxonomy structuring human-AI interaction along a delegation-depth axis.
- **RQ3 (Gap Analysis):** A 19×9 capability-by-SDLC matrix that systematically cross-references supply-side capabilities against demand-side usage, identifying 42 under-evidenced intersections (gaps) and 12 entirely empty intersections.

7.1 The Supply-Demand Asymmetry and Research Agenda

The dominant empirical pattern revealed across this landscape is a stark asymmetry between supply-side saturation and demand-side evidence scarcity. While the reference surveys outline supply-side capabilities harmonized here into 19 unified categories, our analysis demonstrates that only 117 of the 171 possible capability-by-SDLC intersections possess above-threshold empirical support. Furthermore, the inductively derived delegation-depth taxonomy operates orthogonally to prior UI-modality frameworks [27], allowing us to cross-reference interaction modes directly against this supply-side capability matrix.

7.2 Practical Implications

This cross-referencing yields four actionable implications:

- **Organizations:** Can utilize the delegation-depth axis as a strategic self-assessment instrument to map and budget for AI-adoption trajectories.
- **Tool Builders:** Can redirect investments away from saturated coding-stage features toward the 42 identified high-priority integration gaps.
- **Researchers:** Can adopt the proposed seven-Call agenda as a rigorous priority set (Calls 1–6 targeting corpus-internal gaps; Call 7 addressing cross-paradigm tool architectures to resolve the Pro-code-dominance transferability boundary).
- **Educators:** Can scaffold capability-ladder curricula based on progressive delegation depth rather than transient tool popularity.

7.3 From Mapping to Execution (ERP3)

To operationalize these findings, the resulting research agenda transitions directly into a downstream execution phase (ERP3). This phase will deploy a four-instrument methodological package: a practitioner survey [14], lab-controlled evaluations, industry field case studies, and a Multivocal Literature Review (MLR) [12]. Supported by a pre-registered nine-item survey bank (§5.4), this agenda exits the present SMS not as a speculative wish-list, but as a rigorously pre-registered priority set primed for immediate execution.

Acknowledgments

The author would like to thank Professor Ta and Professor Zhu for their expert supervision, constructive feedback, and dedicated mentorship throughout the lifecycle of this project, which significantly shaped the direction of this research.

Declaration of Generative AI Use

During the preparation of this work, the author utilized several generative AI models to assist with specific tasks: Gemini-pro was used to facilitate qualitative analysis, manuscript drafting, and image generation; GPT-5.4 was used to validate analytical outputs; and Claude was used to support code development. The author thoroughly reviewed and refined all AI-assisted materials and takes full responsibility for the content of this published article.

REFERENCES

- [1] Lydia Ajimati and others. 2025. Adoption of low-code and no-code development: A systematic literature review and future research agenda. (seed survey; preprint) (2025).
- [2] Matthew Oladeji Ajimati, Noel Carroll, and Mary Maher. 2025. Adoption of low-code and no-code development: A systematic literature review and future research agenda. *Journal of Systems and Software* 222, (April 2025), 112300. <https://doi.org/10.1016/j.jss.2024.112300>
- [3] Adam Alami, Victor Jensen, and Neil Ernst. 2025. Accountability in code review: The role of intrinsic drivers and the impact of llms. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3721127>
- [4] Nauman Bin Ali and Kai Petersen. 2014. Evaluating strategies for study selection in systematic literature studies. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, September 18, 2014. ACM, Torino Italy, 1–4. <https://doi.org/10.1145/2652524.2652557>
- [5] Jessica Y Bo, Majeed Kazemitabaar, Mengqing Deng, Michael Inzlicht, and Ashton Anderson. 2025. Invisible saboteurs: Sycophantic llms mislead novices in problem-solving tasks. In *ACM conference*, 2025. . <https://doi.org/10.1145/3772318.3791365>
- [6] Yu Cheng, Jieshan Chen, Qing Huang, Zhenchang Xing, Xiwei Xu, and Qinghua Lu. 2024. Prompt sapper: a LLM-empowered production tool for building AI chains. *ACM Transactions on Software Engineering and Methodology* (2024). <https://doi.org/10.1145/3638247>
- [7] D. S. Cruzes and T. Dyba. 2011. Recommended Steps for Thematic Synthesis in Software Engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement*, September 2011. IEEE, Banff, AB, 275–284. <https://doi.org/10.1109/ESEM.2011.36>
- [8] Daniela S. Cruzes, Tore Dybå, Per Runeson, and Martin Höst. 2015. Case studies synthesis: a thematic, cross-case, and narrative synthesis worked example. *Empir Software Eng* 20, 6 (December 2015), 1634–1665. <https://doi.org/10.1007/s10664-014-9326-8>
- [9] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fournay, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent AI systems. In *CHI conference on human factors in computing systems*, 2025. . <https://doi.org/10.1145/3706598.3713581>
- [10] Nat Friedman. 2021. Introducing GitHub Copilot: your AI pair programmer. *The GitHub Blog*. Retrieved April 7, 2026 from <https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/>
- [11] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3712005>
- [12] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106, (February 2019), 101–121. <https://doi.org/10.1016/j.infsof.2018.09.006>
- [13] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, Yinjun Wu, Wentao Zhang, Bin Cui, and Peng Chen. 2025. SiriusBI: a comprehensive LLM-powered solution for data analytics in business intelligence. *Proceedings of the VLDB Endowment* (2025). <https://doi.org/10.14778/3750601.3750610>
- [14] Barbara A. Kitchenham and Shari L. Pfleeger. 2008. Personal Opinion Surveys. In *Guide to Advanced Empirical Software Engineering*, Forrest Shull, Janice Singer and Dag I. K. Sjøberg (eds.). Springer, London, 63–92. https://doi.org/10.1007/978-1-84800-044-5_3
- [15] Barbara Kitchenham, Riallette Pretorius, David Budgen, O. Pearl Brereton, Mark Turner, Mahmood Niazi, and Stephen Linkman. 2010. Systematic literature reviews in software engineering – A tertiary study. *Information and Software Technology* 52, 8 (August 2010), 792–805. <https://doi.org/10.1016/j.infsof.2010.03.006>
- [16] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI agents still need you: Findings from developer-agent collaborations in the wild. <https://doi.org/10.48550/arXiv.2506.12347>

- [17] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild. <https://doi.org/10.48550/arXiv.2506.12347>
- [18] Yvonna S. Lincoln and Egon G. Guba. 1985. *Naturalistic inquiry*. Sage Publications, Beverly Hills, CA.
- [19] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Trans. Softw. Eng. Methodol.* (March 2026), 3796507. <https://doi.org/10.1145/3796507>
- [20] Nesreen Otoum and Nuha Elkhaili. 2026. Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review. *IEEE Access* 14, (2026), 7443–7465. <https://doi.org/10.1109/ACCESS.2026.3652325>
- [21] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3697010>
- [22] Eunil Park. 2025. Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10730-1>
- [23] Kai Petersen, Robert Feldt, Shahid Mujtaba, and Michael Mattsson. 2008. Systematic mapping studies in software engineering. In *Proceedings of the 12th international conference on Evaluation and Assessment in Software Engineering*. 68–77. <https://doi.org/10.5555/2227115.2227123>
- [24] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology* 64, (August 2015), 1–18. <https://doi.org/10.1016/j.infsof.2015.03.007>
- [25] Florian Tambon, Arghavan Moradi-Dakheel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10614-4>
- [26] Christoph Treude and Marco A. Gerosa. 2025. How developers interact with AI: a taxonomy of human-AI collaboration in software engineering. <https://doi.org/10.48550/arXiv.2501.08774>
- [27] Christoph Treude and Marco A. Gerosa. 2025. How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering. <https://doi.org/10.48550/arXiv.2501.08774>
- [28] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI conference on human factors in computing systems extended abstracts*, 2022. . <https://doi.org/10.1145/3491101.3519665>
- [29] Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. 2025. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities. <https://doi.org/10.48550/arXiv.2508.11126>
- [30] Yanlin Wang, WanJun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: survey, landscape, and vision. *Automated Software Engineering* 32, 2 (August 2025). <https://doi.org/10.1007/s10515-025-00544-2>
- [31] Justin D. Weisz, Shraddha Vijay Kumar, Michael Muller, Karen-Ellen Browne, Arielle Goldberg, Katrin Ellisce Heintze, and Shagun Bajpai. 2025. Examining the use and impact of an AI code assistant on developer productivity and experience in the enterprise. In *Extended abstracts of the CHI conference on human factors in computing systems (CHI EA '25)*, 2025. . <https://doi.org/10.1145/3706599.3706670>
- [32] Roel Wieringa, Neil Maiden, Nancy Mead, and Colette Rolland. 2006. Requirements engineering paper classification and evaluation criteria: a proposal and a discussion. *Requirements Eng* 11, 1 (March 2006), 102–107. <https://doi.org/10.1007/s00766-005-0021-6>
- [33] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *CHI conference on human factors in computing systems*, 2022. . <https://doi.org/10.1145/3491102.3517582>

APPENDIX A: PROTOCOL ARTEFACTS

A.1 Search Strings

Six search strings were designed for Scopus, targeting title, abstract, and keywords to span the six scope sub-spaces that motivate the study:

- **S1 — AI coding assistants (adoption-framed).** ("AI coding assistant" OR "AI code generation" OR "LLM-based coding" OR "LLM coding assistant") AND ("usage" OR "adoption" OR "practice" OR "experience")
- **S2 — Named tool mentions (concrete surfaces).** ("GitHub Copilot" OR "ChatGPT" OR "Antigravity" OR "Claude Code" OR "Cursor" OR "Windsurf" OR "Devin" OR "Power Platform" OR

- "n8n" OR "AppSheet" OR "Amazon Q" OR "Codex") AND ("software engineering" OR "software development" OR "application development")
- **S3 — Agentic software engineering (empirical-framed).** ("agentic software engineering" OR "SE agent" OR "coding agent") AND ("empirical" OR "case study" OR "survey" OR "interview")
- **S4 — AI-assisted programming (population-framed).** ("AI-assisted programming" OR "AI pair programming" OR "AI-assisted development") AND ("developer" OR "software engineer" OR "SWE" OR "practitioner" OR "student" OR "citizen developer" OR "business user")
- **S5 — LLMs × SDLC activities.** ("LLM" OR "large language model") AND ("code review" OR "testing" OR "debugging" OR "deployment") AND ("human" OR "user")
- **S6 — Low-code / no-code / citizen-developer AI.** ("low-code" OR "no-code" OR "citizen developer" OR "workflow automation") AND ("AI" OR "LLM" OR "large language model" OR "copilot" OR "agent") AND ("software development" OR "software engineering" OR "application development" OR "app development")

A parallel ACM Digital Library run used ACM-DL syntax for the same six sub-spaces. Snowballing seeds: Wang et al. [29], Wang et al.[30], Otoum & Elkhaili [20], Ajimati et al. [1], and Liu et al. [19].

A.2 Inclusion and Exclusion Criteria

The protocol declares five inclusion criteria and six exclusion criteria. The authoritative source is artifacts/protocol/inclusion_exclusion.md Criteria are enforced across three layers: **Q** = Scopus query (Layer 1 — mechanical filter), **S** = manual title / abstract screening (Layer 2), **P** = post-retrieval script (Layer 3).

Table 4: Inclusion criteria (IC1–IC5)

Code	Rule	Layer
IC1	Publication between 1 January 2022 and the search cut-off (2026-04-13).	Q
IC2	Reports at least one empirical study (survey, interview, case study, experiment, field study, mining study) of human interaction with SE agents, AI coding tools, or AI-enhanced low-code / no-code and workflow-automation platforms used for software or application development.	S
IC3	Peer-reviewed journal article or conference paper (Scopus SRCTYPE journal or conference proceedings; DOCTYPE article or conference paper).	Q
IC4	Written in English.	Q
IC5	At least 4 pages in length.	P

Table 5: Exclusion criteria (EC1–EC6)

Code	Rule	Layer
EC1	The paper only proposes or builds an agent system without reporting usage evidence. Classified as Solution Proposal per Wieringa et al. 2006 and excluded because it contributes to the supply side, not the demand side this SMS maps.	S
EC2	The paper is a benchmark study evaluating model or agent capabilities without human users. Classified as Validation Research per Wieringa et al. 2006 and excluded because it lacks the empirical human-interaction evidence required by IC2.	S
EC3	The paper is outside the software engineering domain. This is a Layer-2 manual screening check; SUBJAREA(COMP) is deliberately not applied at Layer 1 to avoid silently dropping citizen-developer and cross-disciplinary venues.	S
EC4	The paper is a secondary study (existing survey or systematic literature review). DOCTYPE(re) is excluded at Layer 1; manual confirmation at Layer 2. Secondary studies are used as snowball seeds only.	Q + S
EC5	The paper is a short paper (< 4 pages), poster, or extended abstract. Partially filtered at Layer 1 by DOCTYPE exclusions (ed, no, le, sh, er, rt); confirmed by the page-count script at Layer 3.	Q + P

Code	Rule	Layer
EC6	The full text is not retrievable within two weeks of identification. Logged in <code>decision_register.csv</code> with the retrieval attempts and deadline.	P

A.3 Per-criterion exclusion counts (execution-time tally)

The Phase-2 screening engine recorded exclusion rulings under an execution-time labelling convention that mirrored the inclusion criteria: “fails IC_i” was logged as “EC_i” in the PRISMA bookkeeping for five of the six criteria. The tally below preserves those execution-time labels for PRISMA transparency; the right-hand column maps each legacy label to the protocol-defined exclusion in Table 5.

Table 6: Per-criterion exclusion-ruling tally

Execution-time label	Rulings	Share of exclusions	Maps to protocol-defined rule
Legacy EC1 (out-of-window)	1,671	39.9%	Failure of IC1 (publication window)
Legacy EC2 (non-empirical)	588	14.0%	EC1 (Solution Proposal) combined with EC2 (Validation Research); a future re-tally will split this total by Wieringa class
Legacy EC3 (out-of-scope)	1,200	28.6%	EC3 (outside software engineering domain)
Legacy EC4 (non-English)	668	15.9%	Failure of IC4 (English-language requirement)
Legacy EC5 (short-paper, post-IC5)	3	0.1%	EC5 (short paper / poster)
Legacy EC6 (non-primary-study)	58	1.4%	EC4 (secondary study)
Legacy IC1 (restored at Layer 2 after Layer-1 false rejection)	2	0.0%	Layer-1 filter overturned at Layer 2
Total rulings	4,190	100.0%	—

A.4 Modality -- F1–F5 codebook

The five facets are pre-committed at Phase 1. Each facet value has an operational definition, positive and negative exemplars, and a borderline rule.

F1 — Contribution type (Wieringa et al. 2006 six-class vocabulary).

Table 7: F1 - Contribution type

Value	Decision rule
Evaluation Research	The paper investigates an artefact <i>in context</i> (industrial, classroom, field) and evaluates its fit against that context. Default for empirical papers that study a deployed tool.
Validation Research	The paper evaluates an artefact under constructed / controlled conditions (lab experiment, benchmark). Default for controlled experiments and method evaluations.
Solution Proposal	The paper proposes a novel technique with a small proof-of-concept evaluation; the main contribution is the proposal, not the evaluation.
Personal Experience	The paper reports a single practitioner’s or team’s reflective experience without systematic evaluation.
Philosophical	The paper proposes a new conceptual lens.
Opinion	The paper offers a positional argument.

F2 — Research methodology. Seven canonical values: Experiment, Case Study, Mixed, Survey, Mining Study, Interview, Field Study. Mixed is used when two or more primary methods contribute comparable evidentiary weight; the component methods are recorded in the extraction-form notes column.

F3 — Population × Context. Population axis: Professional SWE, Student, Citizen Developer, OSS Contributor, Mixed. Context axis: Industry, Education, OSS, Lab, Mixed. F3 is set to NaN where a paper carries no study-population description.

F4 — SDLC activity. Nine values: Requirements, Design, Coding, Testing, Code Review, Debugging, CI/CD, Documentation, and Project Management. F4 is multi-valued in storage (pipe-separated); a paper describing a tool used for both coding and debugging is legitimate evidence for both activities. Counting-unit disclosures accompany every F4-derived table or figure (distinct papers vs. paper–activity bindings).

F5 — Agent / tool profile. Composite tag with two sub-facets. *Development paradigm* (single-valued): Pro-code, Low-code, No-code. *Interaction modality* (multi-select): Autocomplete, Conversational, IDE-Integrated, Autonomous. Multi-select modality is essential: several mainstream tools span Autocomplete, Conversational, and IDE-Integrated simultaneously.

A.5. Extraction-form schema

Each included paper populates a row of artifacts/extraction/extraction_matrix.csv with bibliographic metadata (paper_id, title, authors, venue, year, DOI), F1–F5 values, sample characterization (participants, context, tool, task), raw verbatim passages describing human–AI interaction (stored separately under artifacts/extraction/raw_passages/), per-paper capability annotations against the 19-capability vocabulary (§A.6), and Mode-A/Mode-B status. Full-text retrieval used a four-tier cascade: Unpaywall → Semantic Scholar → arXiv search → manual acquisition.

A.6 Capability-list harmonization

Nineteen unified capability IDs were built by overlaying the capability vocabularies of the three reference surveys [20, 29, 30] and collapsing near-synonyms. Each capability carries an in_source_survey count in {1, 2, 3} that feeds the RQ3 priority formula.

Table 8: Harmonized capability vocabulary with reference-survey prevalence

Capability ID	Label	In source surveys
CAP_CODEGEN	Code Generation	3
CAP_CODECOMP	Code Completion	3
CAP_PROGREPAIR	Program Repair / Bug Fixing	3
CAP_CODETRANS	Code Translation	3
CAP_CODESUM	Code Summarization	1
CAP_TESTING	Test Generation and Execution	3
CAP_DEBUGGING	Debugging and Fault Localization	3
CAP_CODEREVIEW	Automated Code Review	3
CAP_REFACTORING	Code Refactoring	2
CAP_VULNDET	Vulnerability Detection	2
CAP_DOCGEN	Documentation Generation	3
CAP_REQENG	Requirements Engineering	2
CAP_SYSDESIGN	System Design and Architecture	1
CAP_COMMITMSG	Commit Message Generation	1
CAP_CODESEARCH	Code Search and Retrieval	1
CAP_PLANNING	Task Planning and Decomposition	2
CAP_MULTIAGENT	Multi-Agent Collaboration	2
CAP_CICD	CI/CD and Deployment Automation	2
CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	3

APPENDIX B: RQ1–RQ2 ARTEFACTS

B.1. Capability, modality, and interaction mode - three-way disambiguation

These three terms are distinct and are kept distinct throughout the study to avoid the category error that the prior literature sometimes commits.

- **Capability** (§A.6) is a supply-side tool attribute: what the AI or agent can do (for example, generate code, repair bugs, translate between languages). Each primary study annotates one or more capabilities; the 19-capability vocabulary is harmonized from the three reference surveys.
- **Modality** (§A.4, F5 sub-facet) is a supply-side tool attribute: how the tool surfaces AI capability to the user (Autocomplete, Conversational, IDE-Integrated, Autonomous). F5 modality is multi-select because mainstream tools span multiple modalities simultaneously.
- **Interaction mode** (§B.3) is an inductively derived demand-side human–AI pattern: how the human engages with the AI along a delegation-depth axis (Inline Completion, Conversational Prompting, Visual / Declarative Composition, Review & Validation, Delegated Task Execution). Mode is derived from open coding of passages describing human–AI interaction; it is not a pre-committed facet.

To clarify these conceptual boundaries: capability defines the functional task (what the AI can do), while modality defines the interface mechanism (how that capability is surfaced). In contrast, the distinction between modality and interaction mode separates an intrinsic system attribute from an observed human behavioral pattern. Because these dimensions are orthogonal, a single system modality can support multiple user interaction modes. For instance, a tool featuring a conversational modality can dynamically facilitate both Mode 2 (Conversational Prompting) and Mode 4 (Review & Validation) engagements, depending entirely on whether the practitioner is actively prompting for novel output or critically evaluating a generated response.

B.2. Per-figure image-explanation notes (Figures 2 to 5)

These accompany the main paper’s Figures 2 to 5 with counting-unit disclosures and interpretation caveats.

- **Figure 2 (F2 research-methodology distribution).** This figure quantifies the distinct paper count across empirical research methods (\$N = 640\$). The Mixed category (139 papers; 21.7%) aggregates studies that explicitly combine two or more canonical methods with comparable evidentiary weight. To preserve analytical granularity, the component methods of these mixed studies are maintained discretely within the underlying dataset and can be systematically decomposed upon request.
- **Figure 3 (Year $\times$ SDLC activity $\times$ developer population bubble plot).** This visualization maps the distinct paper count for each year-by-SDLC intersection, faceted by F3 target populations (bubble area denotes volume). The effective sample size for this figure is reduced to 392 (from 640) because 246 papers lack an explicit human-participant description, predominantly algorithmic benchmark evaluations, and are thus excluded. Because SDLC activity is a multi-valued attribute, a single paper addressing multiple phases (e.g., Coding and Debugging) independently increments the count in all applicable cells. Given the pronounced sparsity in the Citizen Developer (19 papers) and OSS Contributor (3 papers) facets, interpretations must rely on aggregate trends across the full visualization rather than drawing inferences from these isolated panels.
- **Figure 4 (SDLC $\times$ year evidence heatmap).** This heatmap illustrates the temporal evolution of research focus across the software lifecycle. Each cell quantifies the distinct paper count for a specific SDLC phase within a given year. The visualization clearly highlights a persistent dominance of code-centric activities (Coding, Testing, and Debugging) throughout the observation window, juxtaposed against a chronic

empirical scarcity in upstream planning (Requirements, Project Management) and downstream operational (CI/CD) activities.

- **Figure 5 (F5 tool profile).** This stacked bar chart maps tool interaction modalities (Autocomplete, Conversational, IDE-Integrated, Autonomous) against development paradigms (Pro-code, Low-code, No-code). The paradigm axis is mutually exclusive and single-valued (figure-plotted distinct-paper counts: Pro-code 537, Low-code 30, No-code 72; total n=639 of 640 corpus papers, the single excluded paper having a NaN modality and therefore no plottable stack segment). While the base paradigm totals are mutually exclusive and single-valued (Pro-code: 538; Low-code: 30; No-code: 72), the internal modality classifications reflect multi-select attributes, acknowledging that modern tools frequently exhibit hybrid interfaces. Consequently, the cumulative count of interaction modalities within a given paradigm stack may exceed the absolute paper count for that paradigm.

B.3. Per-mode operational definitions and paraphrased exemplars

Each mode carries an operational definition, IS / IS-NOT distinguishing criteria, and at least two paraphrased exemplars traced to source papers. Every exemplar is paraphrase-linter clean against the raw passage.

Mode 1: Inline Completion

- **Operational Definition:** The human writes code in-flow while the AI proposes short continuations (ranging from single tokens to a few lines) that the human immediately accepts, modifies, or rejects. The fundamental unit of interaction is the single accept/reject decision; no turn-based dialogue is required.
- **Boundary Criteria:** Includes interactions where the continuation appears inline within the editor, decisions are executed without a separate conversational turn, and the AI takes no multi-step initiative. Excludes iterative prompt refinement (Mode 2) and any instance where the AI executes a task without an immediate token-level decision (Mode 5).
- **Empirical Exemplars:** (1) Vaithilingam et al.'s [28] controlled study of 24 programmers compared an inline Copilot surface—proposing per-line continuations—against a chat-style GPT-3 workflow; the inline accept/reject surface typifies Mode 1. (2) Developers routinely invoking autocomplete to finish the current line during active typing, with acceptance or rejection occurring without opening a chat turn.

Mode 2: Conversational Prompting

- **Operational Definition:** The human articulates intent via natural language, and the AI generates a multi-line response. The human reads, evaluates, and iterates over multiple turns until the output is usable. The fundamental unit of interaction is the turn. Delegation scope is bounded by a single prompt–response cycle, with multiple cycles compounding into a session.
- **Boundary Criteria:** Includes turn-based exchanges via chat or comparable interfaces where the human re-prompts to steer output, and the scope of any single AI action is strictly bounded by the most recent prompt. Excludes inline accept/reject actions (Mode 1), visual/declarative inputs (Mode 3), and multi-step execution without intermediate steering (Mode 5).
- **Empirical Exemplars:** (1) Participants reporting a dominant strategy of authoring natural-language prompts—often embedded as in-file comments—to articulate desired code behavior, refining across turns until the output matched intent. (2) A debugging turn involving pasting a code fragment into a chat interface, appending the observed exception, and requesting a diagnosis and fix; the dialogue extends across subsequent turns if the initial response is incomplete.

Mode 3: Visual / Declarative Composition

- **Operational Definition:** The human composes software by arranging visual, structural, or declarative elements (e.g., drag-and-drop blocks, flowchart nodes, configuration forms, or natural-language-to-workflow pipelines). The AI subsequently generates the underlying implementation (logic, data models, workflow steps) from this declarative specification. The fundamental unit of interaction is the declarative specification itself, not a free-text prompt.
- **Boundary Criteria:** Includes visual or declarative surfaces where the human's input is structural rather than conversational; encompasses Low-code, No-code, and visual-AI-platform evidence. Excludes free-text prompt-and-response interfaces (Mode 2) and code-editor inline completions (Mode 1).
- **Empirical Exemplars:** (1) A prompt-composition tool that surfaces construction through a visual analytics interface, allowing the user to manipulate structural elements (exploration panels, perturbation widgets, test harnesses) rather than authoring chat turns. (2) Structural diagrams (visualized code trees and logic flows) enabling practitioners to compose logic declaratively; the diagram itself acts as the surface of interaction.

Mode 4: Review & Validation

- **Operational Definition:** The AI or the human performs an evaluative reading of code or AI output—detecting hallucinations, assessing quality, localizing defects, or providing/soliciting feedback. The fundamental unit of interaction is the evaluative judgment. The delegation axis interprets this mode bidirectionally as either "AI-as-subject-of-review" or "AI-as-review-provider."
- **Boundary Criteria:** Includes the explicit evaluation of code or AI output for correctness, quality, or trustworthiness, applying to both human-reviewing-AI and AI-reviewing-human directions. Excludes the generation of new code (Modes 1, 2, and 5) and the refinement of a prompt where evaluation remains internal to a conversational turn (Mode 2).
- **Empirical Exemplars:** (1) Evaluators sorting AI-generated unit tests into distinct defect families (unresolved symbols, parameter mismatches, abstract-class instantiations) traceable to hallucinations; the resulting work product is purely evaluative. (2) Reviewers applying an explicit four-axis rubric to AI-produced code (artifact quality, correctness, turnaround speed, and human-baseline comparison), where the interaction unit is the judgment rather than the generation step.

Mode 5: Delegated Task Execution

- **Operational Definition:** The human scopes a broad software engineering task (e.g., generating a test suite, fixing a bug, refactoring a module, or building a workflow), and the AI executes it end-to-end or across multiple sub-steps. The human reviews the output at designated checkpoints or upon completion. The fundamental unit of interaction is the overarching task assignment. (Note: This mode consolidates the provisional HITL Delegation and Autonomous Orchestration categories from the initial protocol).
- **Boundary Criteria:** Includes instances where the AI takes multi-step initiative (e.g., orchestrating an agentic workflow or walking a debugging session), and the human steers primarily at task boundaries rather than per-token. Excludes single-turn prompt-responses (Mode 2) and token-level suggestions (Mode 1).
- **Empirical Exemplars:** (1) An agentic framework decomposing a user-supplied test-generation task into sub-steps, authoring the tests, executing them against the target, and reporting a consolidated result; human intervention occurs strictly at the checkpoint. (2) Weisz et al. [31] reporting a multi-week industrial deployment where a code-assistant agent assumed multi-step tasks (generate, refactor, document) under human checkpoint review; the interaction unit was the holistic assignment.

Residuals (Unclassified Evidence)

- **Operational Definition:** Canonical labels that describe empirical evidence about human–AI interaction rather than documenting a mode-defining interaction episode. This encompasses outcomes of use (192 labels), constraints and barriers (119 labels), meta/reflective commentary (101 labels), and tool affordances (56 labels).
- **Note:** The Residuals layer forms a coherent and distinct analytical sub-structure and is reported jointly alongside the five-mode partition.

B.4 Mode × cross-tabs (summary)

- **Interaction Modes Across the SDLC.** An analysis of row percentages (artifacts/analysis/rq2_variation/mode_x_sdlc.md) reveals that the Coding phase universally dominates across all interaction modes: Mode 1 (80.0%), Mode 2 (59.6%), Mode 3 (81.8%), Mode 4 (52.8%), Mode 5 (57.9%), and Residuals (60.8%). Notably, Mode 5 exhibits the most substantial CI/CD footprint (11 papers; 9.6%), aligning with the delegation of autonomous execution tasks. Conversely, Mode 4 captures the highest share of Code Review activities (25.5%), corroborating its inherently evaluative stance.
- **Interaction Modes and Tool Profiles (F5).** Tool profile distributions (artifacts/analysis/rq2_variation/mode_x_tool_profile.md) are anchored by a denominator of 255, representing the count of distinct papers carrying both an F5 paradigm value and at least one Residuals-classified label, rather than the total count of all passage-bearing papers. Within this subsample, restricted to the Residuals row (255 papers), the Pro-code paradigm dominates at 87.5%. No-code papers (22 of 255) are distributed across Modes 2 (11), 3 (1), 4 (11), and 5 (10). Low-code papers (10 of 255) are similarly split among Modes 2 (3), 3 (2), 4 (3), and 5 (4). Mode 3 (Visual / Declarative Composition) is unique in capturing a substantial Low-code share (18.2% of its 11-paper subsample, compared to $\leq 3.5\%$ across all other modes), which directly reflects its underlying visual and declarative construct. Furthermore, Mode 3’s No-code share (9.1%) is comparable to that of Modes 2 (11.7%), 4 (10.4%), and 5 (8.8%), indicating that No-code evidence is dispersed across multiple interaction paradigms rather than isolated to a single mode.
- **Demographics and Contextual Distribution (F3).** Population and context distributions (artifacts/analysis/rq2_variation/mode_x_population_context.md; Residuals-row denominator $N = 255$) indicate a heavy concentration in professional and academic settings. The Professional SWE / Industry intersection accounts for 21.2% (54 papers), followed by Student / Education at 16.9% (43 papers), Mixed / Lab at 10.6% (27 papers), and Professional SWE / Lab at 5.1% (13 papers). In stark contrast, Citizen-Developer and OSS-Contributor populations jointly contribute a mere 2.7% (7 papers). This density remains too thin to support robust, mode-level prevalence claims for these specific cohorts.

APPENDIX C: RQ3 AND TRUSTWORTHINESS ARTEFACTS

C.1 Objectives × reference surveys — novelty-comparison matrix (4 × 3)

Every cell record whether a given research objective is delivered by a given reference survey. The matrix documents zero overlap at any cell; cells are populated by inspecting each survey’s scope statement, research-question set, and findings section:

Table 9: Novelty-comparison matrix

Objective \ Reference survey	Wang et al. [29]	Wang et al. [30]	Otoum & Elkhaili [20]
(i) 2022–2026 empirical landscape along five facets	—	—	—

Objective \ Reference survey	Wang et al. [29]	Wang et al. [30]	Otoun & Elkhaili [20]
(ii) 5-mode interaction taxonomy along a delegation-depth axis	—	—	—
(iii) 19×9 capability $\times$ SDLC gap matrix (supply vs demand)	—	—	—
(iv) Wieringa-typed 7-call research agenda (Calls 1–6 corpus-internal, covering 42 gaps + 12 empty cells; Call 7 cross-paradigm taxonomy extension addressing the Pro-code-dominance transferability boundary)	—	—	—

All twelve cells are empty: no reference survey delivers any of the four objectives. The novelty claim is therefore not a marginal improvement over prior work; it is a demand-side mapping that sits adjacent to the three supply-side surveys.

C.2 Eight transferability boundaries

Drawn from Lincoln & Guba [18] and adapted to SE by Cruzes & Dybå [7]. Each boundary names (a) the scope limit, (b) how it was introduced, (c) which findings it constrains.

1. Time window (2022–2026). Evidence concentration is 2022–2026 only; trends do not extrapolate pre-2022. Limits: RQ1 landscape, RQ2 mode prevalence.
2. Venue set (Scopus + ACM DL + five seed surveys; no grey literature). Any practice reported only in vendor blog posts or industry reports is invisible. Limits: RQ1 coverage.
3. Language (English-only). Region-specific practice published in non-English venues is out of scope. Limits: RQ1 regional coverage.
4. Paradigm skew (Pro-code dominance). 84.1% of F5-profiled papers are Pro-code. Limits: RQ2 Mode 3 prevalence; RQ3 per-paradigm gap inference (Low-code 30 papers, No-code 72 papers).
5. F3 population coverage (61.6% canonical). 246 of 640 papers carry F3 = NaN. Limits: RQ1 population-faceted figures; RQ2 Mode $\times$ population cross-tabs.
6. Mode 1 thin evidence (4 of 707 canonical labels). Publication bias, not usage sparsity. Limits: any Mode 1 prevalence claim.
7. Mode-A / Mode-B coding coverage (290 / 350). Abstract-only papers (Mode B) cannot support inductive passage coding. Limits: RQ2 derivation denominator. ERP3 interviews are the remedy.
8. ERP3 hand-off scope. The research agenda is a pre-registered priority set, not a validated finding. Limits: agenda authority; ERP3 may validate, extend, or refute.

C.3 P20 / P25 / P33 / P40 threshold sensitivity

Table 10: RQ3 gap-rule threshold sensitivity

Percentile	Threshold	Gap cells	Empty cells	Normal cells	Δ vs P25
P20	2.000	42	12	117	0
P25 (primary)	2.000	42	12	117	—
P33 (sensitivity)	3.140	53	12	106	+11
P40	5.000	72	12	87	+30

Threshold Sensitivity and Conservative Bias. As detailed in Appendix C.3, the 20th (P20) and 25th (P25) percentiles yield identical threshold values for this corpus (2.000) due to the right-skewed nature of the non-zero cell distribution. The count of entirely empty cells ($N = 12$) naturally remains invariant across all percentile adjustments. Crucially, the gap classification demonstrates monotonic stability: no cell flagged within the primary P25 gap set loses

its classification at higher thresholds, establishing a strict subset sequence ($P25 \subseteq P33 \subseteq P40$). Elevating the threshold to P33 introduces 11 additional cells to the gap set. Notably, this expansion captures emerging intersections in agentic capabilities; specifically, CAP_MULTIAGENT applied to Project Management, and CAP_PLANNING applied to CI/CD and Code Review. The omission of these specific gaps under the primary P25 rule highlights a conservative bias inherent in the strict primary threshold.

C.4 Full 42-row prioritized gap table

Ordered by priority (desc), ties broken by capability-ID then SDLC activity.

Table 11: Full 42-row prioritized gap table

Rank	Capability	Label	SDLC	Evidence	Priority	Bucket
1	CAP_CODECOMP	Code Completion	Design	1	1.50	design_gap
2	CAP_CODECOMP	Code Completion	Requirements	1	1.50	design_gap
3	CAP_CODETRANS	Code Translation	CI/CD	1	1.50	organizational_readiness_barrier
4	CAP_CODETRANS	Code Translation	Project Management	1	1.50	organizational_readiness_barrier
5	CAP_CODETRANS	Code Translation	Requirements	1	1.50	design_gap
6	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Code Review	1	1.50	adoption_lag
7	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Documentation	1	1.50	adoption_lag
8	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Project Management	1	1.50	adoption_lag
9	CAP_MULTIAGENT	Multi-Agent Collaboration	CI/CD	1	1.00	adoption_lag
10	CAP_MULTIAGENT	Multi-Agent Collaboration	Code Review	1	1.00	adoption_lag
11	CAP_MULTIAGENT	Multi-Agent Collaboration	Design	1	1.00	design_gap
12	CAP_VULNDET	Vulnerability Detection	CI/CD	1	1.00	organizational_readiness_barrier
13	CAP_VULNDET	Vulnerability Detection	Project Management	1	1.00	organizational_readiness_barrier
14	CAP_CODESEARCH	Code Search & Retrieval	Code Review	1	0.50	organizational_readiness_barrier
15	CAP_CODESEARCH	Code Search & Retrieval	Debugging	1	0.50	organizational_readiness_barrier
16	CAP_CODESEARCH	Code Search & Retrieval	Documentation	1	0.50	organizational_readiness_barrier
17	CAP_CODESEARCH	Code Search & Retrieval	Testing	1	0.50	organizational_readiness_barrier
18	CAP_CODESUM	Code Summarisation	Requirements	1	0.50	design_gap
19	CAP_COMMITMSG	Commit Message Generation	CI/CD	1	0.50	organizational_readiness_barrier

Rank	Capability	Label	SDLC	Evidence	Priority	Bucket
20	CAP_COMMITMSG	Commit Message Generation	Code Review	1	0.50	organizational_readiness_barrier
21	CAP_COMMITMSG	Commit Message Generation	Debugging	1	0.50	organizational_readiness_barrier
22	CAP_COMMITMSG	Commit Message Generation	Project Management	1	0.50	organizational_readiness_barrier
23	CAP_COMMITMSG	Commit Message Generation	Testing	1	0.50	organizational_readiness_barrier
24	CAP_CICD	CI/CD & Deployment Automation	Code Review	2	0.00	organizational_readiness_barrier
25	CAP_CICD	CI/CD & Deployment Automation	Design	2	0.00	design_gap
26	CAP_CICD	CI/CD & Deployment Automation	Documentation	2	0.00	organizational_readiness_barrier
27	CAP_CICD	CI/CD & Deployment Automation	Project Management	2	0.00	organizational_readiness_barrier
28	CAP_CODEREVIEW	Automated Code Review	Project Management	2	0.00	organizational_readiness_barrier
29	CAP_CODESUM	Code Summarisation	Code Review	2	0.00	organizational_readiness_barrier
30	CAP_CODESUM	Code Summarisation	Design	2	0.00	design_gap
31	CAP_CODETRANS	Code Translation	Design	2	0.00	design_gap
32	CAP_CODETRANS	Code Translation	Testing	2	0.00	organizational_readiness_barrier
33	CAP_COMMITMSG	Commit Message Generation	Coding	2	0.00	organizational_readiness_barrier
34	CAP_COMMITMSG	Commit Message Generation	Documentation	2	0.00	organizational_readiness_barrier
35	CAP_MULTIAGENT	Multi-Agent Collaboration	Documentation	2	0.00	adoption_lag
36	CAP_PROGREPAIR	Program Repair / Bug Fixing	Design	2	0.00	design_gap
37	CAP_PROGREPAIR	Program Repair / Bug Fixing	Requirements	2	0.00	design_gap
38	CAP_REQENG	Requirements Engineering	CI/CD	2	0.00	design_gap
39	CAP_REQENG	Requirements Engineering	Code Review	2	0.00	design_gap

Rank	Capability	Label	SDLC	Evidence	Priority	Bucket
40	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Design	2	0.00	design_gap
41	CAP_SYSDSIGN	System Design & Architecture	Project Management	2	0.00	design_gap
42	CAP_VULNDET	Vulnerability Detection	Documentation	2	0.00	organizational_readiness_barrier

C.5 Research agenda — per-call detail

We define a structured research agenda comprising seven formalized Calls to Action. Each Call operates as a pre-registered proposal for a future primary study, specifying a methodological classification drawn from Wieringa’s [32] taxonomy, a target gap set (or scope extension), a research-question template, and a targeted population fit. Calls 1 through 6 are corpus-internal, explicitly operationalizing the 42 identified gaps and 12 empty cells: Calls 1–5 target the 13 highest-priority gaps (priority ≥ 1.00), while Call 6 serves as an integrative Evaluation Research initiative covering a cluster of ten moderate-priority (priority = 0.50) knowledge-management gaps. Call 7 extends the agenda beyond the internal gap matrix to address the Pro-code-dominance transferability boundary; it is framed as a downstream taxonomy construction and validation study rather than a standard gap-targeted investigation.

Call 1: AI Code Completion in Early-Lifecycle Phases

- **Methodological Class:** Evaluation Research.
- **Targeted Gaps:** Ranks 1–2 (Two priority-1.50 gaps: $\text{CAP_CODECOMP} \times \{\text{Design, Requirements}\}$).
- **Research Question Template:** How do practitioners utilize code-completion tools during requirements elicitation and software design, and how do these usage patterns diverge from the coding-stage applications that currently dominate the literature?
- **Population Fit:** Professional software engineers and business analysts within industry contexts.

Call 2: AI Code Translation Beyond the Coding Boundary

- **Methodological Class:** Validation Research (at Requirements); Evaluation Research (at CI/CD and Project Management).
- **Targeted Gaps:** Ranks 3–5 (Three priority-1.50 gaps: $\text{CAP_CODETRANS} \times \{\text{CI/CD, Project Management, Requirements}\}$).
- **Research Question Template:** How do legacy-modernization initiatives integrate AI code translation into migration pipelines and project management, and under what conditions does automated translation reliably preserve specification intent?
- **Population Fit:** Enterprise modernization teams within industry contexts.

Call 3: Agentic Self-Reflection Outside of Coding

- **Methodological Class:** Validation Research.
- **Targeted Gaps:** Ranks 6–8 (Three priority-1.50 gaps: $\text{CAP_SELFREFLECT} \times \{\text{Code Review, Documentation, Project Management}\}$).
- **Research Question Template:** To what extent does agentic self-reflection yield measurably superior outcomes for code-review synthesis, documentation generation, and project-management task decomposition compared to single-pass baselines?

- **Population Fit & Constraints:** Designed for researcher-led lab studies or industry pilot programs. Following Kumar et al. [16], studies must pre-register a strict falsifiability constraint: self-reflection is credited only if its output-quality advantage persists under user-disagreement conditions, actively controlling for sycophancy and overconfidence failure modes.

Call 4: Multi-Agent Orchestration at Under-Studied SDLC Intersections

- **Methodological Class:** Validation Research.
- **Targeted Gaps:** Ranks 9–11 (Three priority-1.00 gaps: $CAP_MULTIAGENT \times \{CI/CD, \text{Code Review, Design}\}$).
- **Research Question Template:** How effectively do multi-agent orchestration frameworks perform as CI/CD pipeline coordinators, collaborative code-review committees, and design-stage ideation partners, and which coordination patterns exhibit the greatest robustness?
- **Population Fit & Constraints:** Targets researcher-led lab studies or early-adopter industry partnerships. Studies must pre-register two falsifiable metrics from Kumar et al. [16]: the unsolicited-agent-action rate (per prompt-turn) to measure coordination quality, and the success differential between one-shot versus incremental coordination patterns across the three target SDLC intersections.

Call 5: AI Vulnerability Detection Beyond Coding

- **Methodological Class:** Evaluation Research.
- **Targeted Gaps:** Ranks 12–13 (Two priority-1.00 gaps: $CAP_VULNDET \times \{CI/CD, \text{Project Management}\}$).
- **Research Question Template:** How is AI-augmented vulnerability detection embedded within CI/CD pipelines and enterprise security management, and which organizational factors best predict the effective triage of AI-generated security alerts?
- **Population Fit & Context:** Enterprise security teams within industry contexts. This Call is empirically supported by Gao et al. [11], who identify security-alert triage as a critical organizational load-bearing challenge in the LLM era.

Call 6: Knowledge Management in Non-Native Activities

- **Methodological Class:** Evaluation Research.
- **Targeted Gaps:** A cluster of ten priority-0.50 gaps spanning $CAP_CODESEARCH \times \{\text{Code Review, Debugging, Documentation, Testing}\}$, $CAP_CODESUM \times \{\text{Requirements}\}$, and $CAP_COMMITMSG \times \{CI/CD, \text{Code Review, Debugging, Project Management, Testing}\}$.
- **Research Question Template:** Outside the primary coding flow, when and how do practitioners invoke AI-driven code search, summarization, and commit-message generation? Crucially, what marginal value or friction do these tools introduce at critical decision points in review, testing, debugging, and deployment?
- **Population Fit:** Mixed-population field studies encompassing professional developers, open-source maintainers, and business analysts.

Call 7: Taxonomy of AI-Agent Architectures Beyond Pro-Code Environments

- **Methodological Class:** Evaluation Research, preceded by a Multivocal Literature Review (MLR) [12]. The MLR is a necessary prerequisite because critical architectural evidence (vendor documentation, platform frameworks, and grey-literature practitioner reports) remains severely underrepresented in peer-reviewed venues.

- Targeted Focus: Addresses an unranked matrix gap motivated entirely by transferability boundary IV (§C.2: Pro-code dominance at 84.1%) and practitioner-reported scenarios logged as hypotheses in the decision register.
- Research Question Template: Along which architectural axis—orthogonal to the delegation-depth axis defined in §4.2—do AI-agent tools partition in practice? How do these positions map to user roles (developer, citizen developer, researcher, end-user), triggering domains (business-process vs. SE-lifecycle), and cross-paradigm integrations?
- Hypothesis Testing: Candidate categories (e.g., generalist AI authoring IDEs, horizontal workflow orchestrators, vertical SaaS with embedded AI, agent-construction frameworks, and deployed personal AI assistants) must be treated as falsifiable hypotheses. The final architectural taxonomy will be established empirically via the MLR and a companion practitioner survey, not assumed a priori.
- Population Fit & Instruments: Requires a mixed-population approach with a deliberate over-sampling of non-Pro-code demographics to explicitly counter corpus boundary IV. The execution package includes Garousi et al.'s MLR guidelines [12], a platform-documentation-analysis protocol, and a practitioner survey [14].

Empty-Cell Appendix and Interpretive Boundary

The 12 matrix intersections lacking empirical evidence are systematically partitioned into two distinct categories based on semantic plausibility. Seven cells are designated as candidate exploratory studies where the capability-by-activity pairing remains theoretically sound (e.g., CAP_CODESUM applied to Project Management for stakeholder reporting, or CAP_CODESEARCH in CI/CD for artifact discovery). Conversely, five cells are classified as architecturally inapplicable (e.g., CAP_COMMITMSG mapped to Requirements or Design, as commit messages intrinsically presuppose existing code artifacts). The authoritative classification for all 12 cells is maintained in artifacts/analysis/research_agenda.md, and this 7-to-5 split is formally logged in the decision register as a singular interpretive judgment. Consequently, architectural inapplicability is reported as a conclusive interpretive finding, not as a target for future research.

C.6 Decision-register schema and sample slice

The Decision Register and Audit Trail. To ensure strict methodological transparency, we maintained a decision register: an append-only audit log documenting every non-trivial classification judgment. The underlying schema systematically captures the timestamp, phase, paper_id, decision, rule_applied, rationale, and rater_initials. To preserve the integrity of the audit trail, records are never modified in place; instead, corrections are strictly appended as new rows (flagged with the decision revise) that explicitly reference the preceding entry.

Phase Distribution and Longitudinal Tracking. At the initial manuscript-draft freeze (April 20, 2026), the register contained 397 distinct rows distributed across five methodological phases: Phase 1 (2), Phase 2 (24), Phase 3 (325), Phase 4 (4), and Phase 5 (42). The heavy concentration in Phase 3 is predominantly driven by full-text F1 revisions (323 rows). Conversely, Phase 5 consists of single, per-gap hypothesis entries (recorded as hypothesis_for_erp3) that map qualitative bucket labels to the identified matrix gaps.

C.7 Data Provenance — source map for every load-bearing numeric claim

Data Provenance Registry. This section serves as the authoritative registry for all quantitative data cited throughout this ERP. Each entry systematically documents a specific claim, its exact numerical value, the source file that authoritatively generates or houses that value, and the applied verification method. To optimize readability while preserving strict traceability, inline data pointers in the main text are provided strictly upon first appearance; all

subsequent references rely on this Data Provenance table as the canonical back-reference. All specified file paths are relative to the root directory of the replication repository.

Table 12: Corpus and extraction counts

Claim	Value	Source	Verification
Primary empirical studies (total corpus)	640	artifacts/screening/included_set.csv	Row count
Papers with full-text retrieved	258	artifacts/extraction/retrieval_status.csv	Filter status == retrieved
Papers in Mode A (full-text, passage-bearing)	290	artifacts/extraction/extraction_status.csv	Filter mode == A
Papers in Mode B (abstract-only)	350	artifacts/extraction/extraction_status.csv	Filter mode == B
Pass-1 open codes	1,016	artifacts/extraction/open_codes_pass1.csv	Row count
Canonical labels after Pass-2 clustering	707	artifacts/synthesis/consolidated_codes.csv	Row count
Capability annotations (total)	776	artifacts/extraction/capability_annotation_s.csv	Row count
Distinct papers carrying capability annotations	384	artifacts/extraction/capability_annotation_s.csv	Distinct paper_id count

Table 13: Facet distributions (F1–F5)

Claim	Value	Source	Verification
F2 Experiment	249 (38.9%)	artifacts/extraction/extraction_matrix.csv	f2_research_methodology == Experiment; denominator 640
F2 Mixed	139 (21.7%)	same	same
F2 Case Study	92 (14.4%)	same	same
F2 Interview	20 (3.1%)	same	same
F2 Field Study	14 (2.2%)	same	same
F3 Student	133	artifacts/extraction/extraction_matrix.csv	f3_population == Student
F3 Professional SWE	128	same	f3_population == Professional SWE
F3 Mixed	111	same	f3_population == Mixed
F3 coverage (non-N/A)	61.6%	same	Count non-N/A ÷ 640 = 394 ÷ 640
F5 Pro-code	538 (84.1%)	artifacts/extraction/extraction_matrix.csv	f5_tool_paradigm == Pro-code; denominator 640
F5 No-code	72 (11.3%)	same	f5_tool_paradigm == No-code
F5 Low-code	30 (4.7%)	same	f5_tool_paradigm == Low-code

Table 14: Gap matrix and priority rankings

Claim	Value	Source	Verification
Gap-matrix total cells (19 × 9)	171	artifacts/analysis/rq3_gap_matrix/gap_matrix.csv	Row count
Normal cells (evidence > P25)	117 (68.4%)	same	Filter is_gap == False AND is_empty == False

Claim	Value	Source	Verification
Gap cells (evidence $\leq$ P25, > 0)	42 (24.6%)	same	Filter <code>is_gap == True</code>
Empty cells (evidence = 0)	12 (7.0%)	same	Filter <code>is_empty == True</code>
P25 threshold (non-zero cells)	2.0	<code>artifacts/analysis/prioritised_gaps.md</code>	Header block
Maximum cell	CAP_CODEGE $N \times \text{Coding} = 156$	<code>artifacts/analysis/rq3_gap_matrix/gap_matrix.csv</code>	<code>max(evidence_count)</code>
CI/CD activity — gaps	5 of 42	same	Filter <code>sdlc_activity == CI/CD AND is_gap == True</code>
CI/CD activity — empty	4 of 12	same	Filter <code>sdlc_activity == CI/CD AND is_empty == True</code>
Priority-1.50 gaps (ranks 1–8)	8	<code>artifacts/analysis/prioritised_gaps.md</code>	Ranked list rows 1–8
Priority-1.00 gaps (ranks 9–13)	5	same	Ranked list rows 9–13
Priority ≥ 1.00 gaps (total)	13	same	Sum of priority-1.50 + priority-1.00
Priority-0.50 gaps (ranks 14–23)	10	same	Ranked list rows 14–23
Priority-0.00 gaps (ranks 24–42, borderline at P25)	19	same	Ranked list rows 24–42
Organisational-readiness bucket	22 of 42 (52.4%)	<code>decision_register.csv</code>	Phase-5 <code>hypothesis_for_erp3</code> rationale filter
Design-gap bucket	14 of 42 (33.3%)	same	same
Adoption-lag bucket	6 of 42 (14.3%)	same	same
Gap-matrix total cells (19 $\times$ 9)	171	<code>artifacts/analysis/rq3_gap_matrix/gap_matrix.csv</code>	Row count

Table 15: Interaction-mode taxonomy counts

Claim	Value	Source	Verification
Mode 1 Inline Completion canonical labels	4 (0.6%)	<code>artifacts/synthesis/interaction_taxonomy.md</code>	Mode-layer tally; denominator 707
Mode 2 Conversational Prompting	69 (9.8%)	same	same
Mode 3 Visual / Declarative Composition	9 (1.3%)	same	same
Mode 4 Review & Validation	82 (11.6%)	same	same
Mode 5 Delegated Task Execution	75 (10.6%)	same	same
Residuals	468 (66.2%)	same	same
Mode-layer saturation — no new modes in final	29 of 290 papers	<code>artifacts/synthesis/saturation_data_mode.csv</code>	Last 10% of processing-order sequence; verified in

Claim	Value	Source	Verification
Canonical merge cosine threshold	0.35	artifacts/protocol/extraction_schema.md	saturation_report_mod e.md Protocol parameter

Table 16: Trustworthiness, audit, and transferability metrics

Claim	Value	Source	Verification
Paraphrase-linter violations	0 of 6,295 15-word substrings	code/dod_checks.py::check_phase7_task7_1	Most recent dispatcher run
Decision-register rows (at manuscript draft)	397	decision_register.csv	Row count at freeze timestamp 2026-04-20
Decision-register rows by phase	P1 = 2; P2 = 24; P3 = 325; P4 = 4; P5 = 42	same	Group-by phase on frozen slice
Transferability boundary iv — Pro-code dominance	84.1% of F5-profiled papers	artifacts/extraction/extraction_matrix.csv	Same cell as F5 Pro-code above
Transferability boundary v — partial F3 coverage	61.6% non-N/A	same	Same cell as F3 coverage above
Transferability boundary vi — Mode 1 thin evidence	4 of 707 canonical labels	artifacts/synthesis/interaction_taxonomy.md	Same cell as Mode 1 above
Transferability boundary vii — Mode-A/B split	290 / 350	artifacts/extraction/extraction_status.csv	Same cells as Mode A / Mode B above
Paraphrase-linter violations	0 of 6,295 15-word substrings	code/dod_checks.py::check_phase7_task7_1	Most recent dispatcher run

C.8. Methodology anchor traceability

As outlined in Section 2.4, five methodological sources inform the nine foundational design decisions of this study. The following table maps each of these decisions to its corresponding design specification, implementation code, and output artifacts. To ensure clarity, each row represents a distinct methodological decision rather than an individual source paper. In cases where a decision lacks a dedicated computational design document, such as the four analytical trustworthiness criteria, the design specification cell is marked accordingly.

Table 17: Methodology anchor traceability

Methodology Anchor	Methodology	Purpose	Design	Code	Artifacts
SMS: 5-facet classification scheme (F1–F5)	Petersen et al. 2008, 2015[23, 24]	Pre-registered facet scheme — Contribution Type (F1), Research Methodology (F2), Population and Context (F3), SDLC Activity	design/3_2_extraction.md, design/2_5_1_screening_harness.md	code/screening_harness.py, code/extraction.py	artifacts/protocol/codebook.md (F1–F5 operational definitions); artifacts/extraction/extraction_matrix.csv (640 papers × facets); manuscript/tables/wieringa_f

Methodology Anchor	Methodology	Purpose	Design	Code	Artifacts
		(F4), Agent / Tool Profile (F5) — applied consistently across every corpus paper.			1_distribution.md, corpus_descriptives.md
SMS: PRISMA attrition flow	Petersen et al. 2008, 2015[23, 24]	Transparent record of search → dedup → IC5 filter → title / abstract screening → snowball → final inclusion.	design/2_7_prisma_builder.md	code/prisma_builder.py	artifacts/screening/prisma_flow.md, GeminiBeautified_PrismaFlow.png; manuscript/tables/prisma_counts.md, exclusion_counts_by_criterion.md; manuscript/figures/prisma_flow.png
SMS: Visualisation conventions (§4.1)	Petersen et al. 2008, 2015[23, 24]	Petersen-standard RQ1 visualisations — year × SDLC bubble plot, F2 methodology bar chart, F5 tool-profile stacked bar, year × SDLC heatmap.	design/5_1_analysis_rq1.md	code/analysis_rq1.py	artifacts/analysis/rq1_landscape/year_sdlc_bubble.png, f2_method_bar.png, f5_tool_profile_stacked.png, sdlc_year_heatmap.png (each with a companion *.meta.json and *_imgexplain.md); mirrored in manuscript/figures/
5-step thematic synthesis → RQ2 taxonomy	Cruzes & Dybå 2011, 2015[7, 8]	Inductively derive a mode-level taxonomy from 1,016 pass-1 codes → 707 canonical labels → 5 modes, with a saturation stopping rule.	design/4_1_coding_consolidate.md, design/4_2_interaction_taxonomy.md, design/4_2_taxonomy_classify.md, design/4_2_taxonomy_classify_llmassist.md, design/4_2_residuals_subcategorise.md, design/4_2_taxonomy_render.md	code/coding_consolidate.py, code/taxonomy_classify.py, code/taxonomy_classify_llmassist.py, code/taxonomy_render.py, code/saturation_report.py	artifacts/synthesis/consolidated_codes.csv, taxonomy_classifications.csv, interaction_taxonomy.md, saturation_data_mode.csv, saturation_report_mode.md, saturation_curve_mode.png
Extraction–synthesis separation	Cruzes & Dybå 2011, 2015[7, 8]	Keep stable descriptive coding (extraction layer) separate from	design/3_2_extraction.md (extraction	code/extraction.py (extraction); code/coding_consolidate.py +	Extraction layer: artifacts/extraction/extraction_matrix.csv, open_codes_pass1.csv,

Methodology Anchor	Methodology	Purpose	Design	Code	Artifacts
		derived higher-order themes (synthesis layer) so extraction can be re-run without contaminating synthesis.	layer); design/4_*(synthesis layer)	code/taxonomy_classifier.py (synthesis)	capability_annotations.csv, raw_passages/*.md. Synthesis layer: artifacts/synthesis/*
F1 contribution-type vocabulary (six Wieringa classes)	Wieringa et al. 2006[32]	Classify each paper as Evaluation Research, Validation Research, Solution Proposal, Philosophical, Opinion, or Personal Experience.	design/2_5_1_screening_harness.md, design/3_2_extraction.md	code/screening_harness.py, code/extraction.py	artifacts/protocol/codebook.md (F1 operational definitions); artifacts/extraction/extraction_matrix.csv (f1_contribution_type column); manuscript/tables/wieringa_f1_distribution.md
Evaluation / Validation Research distinction — research agenda	Wieringa et al. 2006[32]	Type every research-agenda Call as Evaluation Research or Validation Research; each Call names its class in the per-Call record.	design/5_3_prioritize_gaps.md	code/prioritize_gaps.py	artifacts/analysis/prioritised_gaps.md, artifacts/analysis/research_agenda.md (per-Call records); manuscript/draft_v2.md §5.3; manuscript/appendix_v2.md §C.5
Backward snowballing (Phase 2)	Ali & Petersen 2014 [4]	Extract references from the five seed surveys and check against the existing corpus. Phase-2 result: 778 references extracted, 40 overlap with corpus, zero new inclusions.	design/2_6_snowball.md	code/snowball.py	artifacts/search/raw/snowball_seeds_refs.csv
Four qualitative-research criteria (credibility, confirmability, dependability, transferability)	Lincoln & Guba 1985, adapted to software engineering by Cruzes & Dybå 2011[7, 18]	Structure the trustworthiness audit in §6. Each criterion has its own audit artefact with specific operational checks.	No dedicated design document (criteria are analytical rather than computational)	code/dod_checks.py (dispatcher), code/paraphrase_linter.py (credibility), code/trace_checker.py (confirmability), code/saturation_report.py (dependability)	artifacts/trustworthiness/credibility_statement.md, confirmability_annex.md, dependability_audit.md, transferability.md, threshold_sensitivity.md